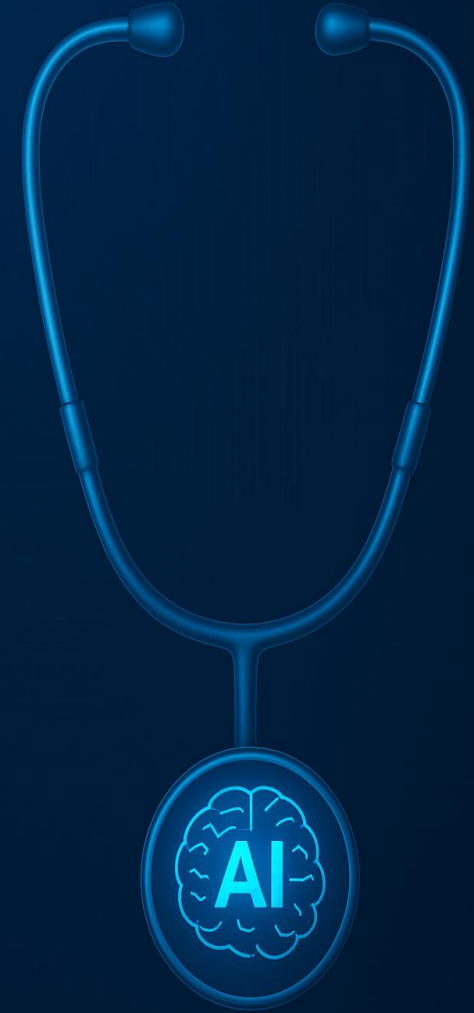


Cours par Mohamad Forouzanfar

L'IA DANS LES SOINS DE SANTÉ

GTS880

Sujets spéciaux en technologies de la santé



Cours 3

ESTIMATION ET CARACTÉRISATION DES PARAMÈTRES LIÉS À LA SANTÉ (BASES DE L'IA)

Rappel :

L'EXAMENS INTRA

LES EXAMENS PORTENT DAVANTAGE SUR LES CONCEPTS QUE SUR LES DÉFINITIONS OU LES MATHÉMATIQUES AVANCÉES.

POUR LES DEUX PREMIERS COURS, NOUS NOUS CONCENTRERONS SUR L'ORIGINE ET LA CRÉATION DES SIGNAUX PHYSIQUES, LES TECHNIQUES PERMETTANT DE LES MESURER ET DE LES CARACTÉRISER, AINSI QUE L'APPLICATION DE L'IA DANS CE DOMAINE.

Objectifs

- Comprendre les bases de l'apprentissage automatique et la terminologie courante
- Apprendre l'analyse de régression linéaire
- Apprendre l'optimisation d'une fonction de coût
- Se familiariser avec le perceptron multicouche (MLP) et l'algorithme de rétropropagation
- Apprendre à concevoir une tâche de ML pour l'estimation ou la caractérisation des paramètres de santé
- Étude de cas : Estimation de la pression artérielle à partir du PPG

Intelligence Artificielle (IA)

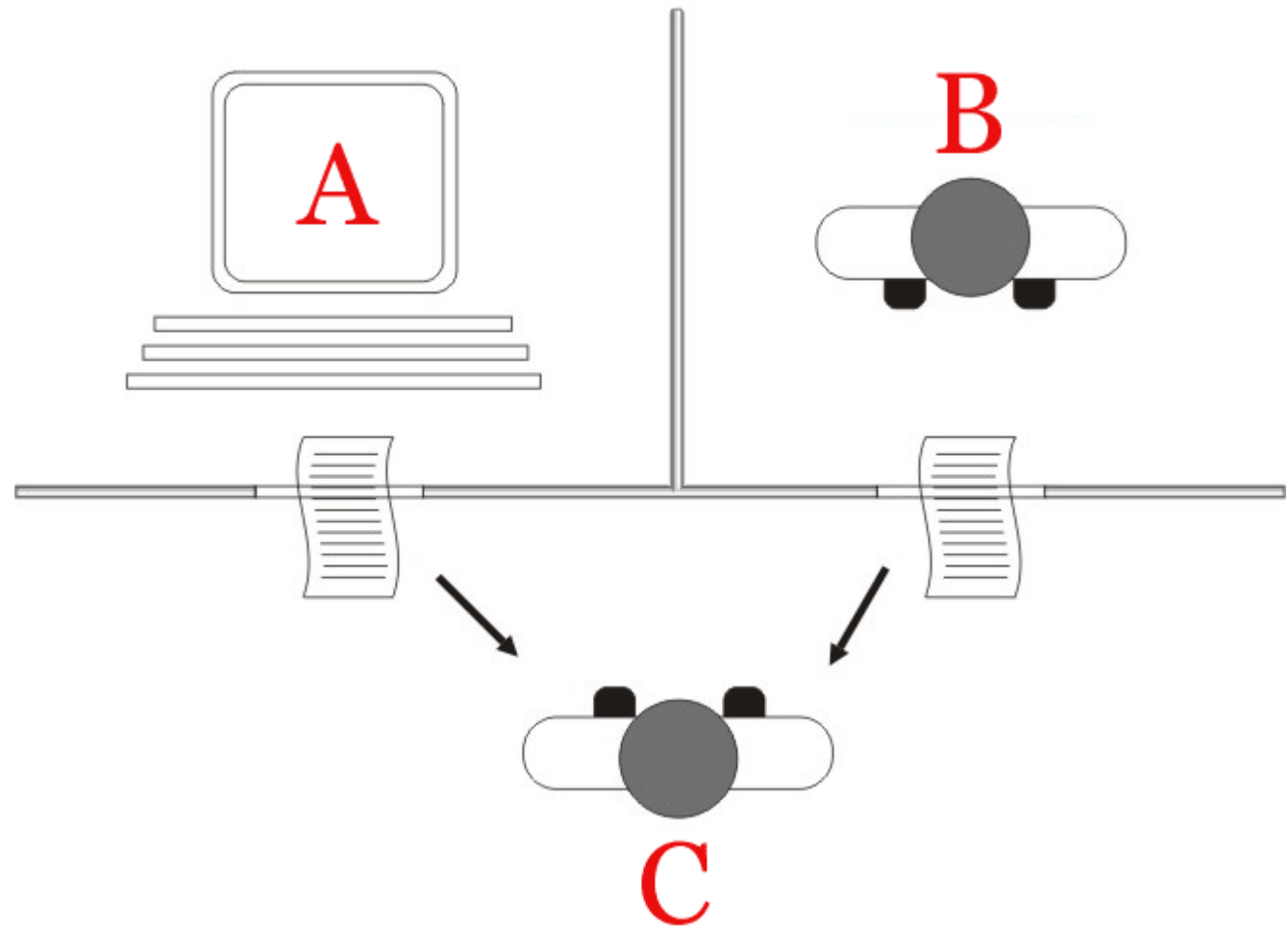
La naissance de l'intelligence artificielle peut être attribuée à une question posée par Alan Turing, souvent considéré comme le "père de l'informatique", en 1950.

Dans un ouvrage fondamental, il s'interroge : **"Les machines peuvent-elles penser ?"** et propose un critère pour évaluer cette capacité, connu sous le nom de **Test de Turing**.

Ce test consiste en une interaction entre un interrogateur humain, un autre humain et un ordinateur. L'interrogateur pose des questions sans savoir qui est qui et doit identifier l'humain parmi les répondants.

L'interrogateur doit décider quel répondant est humain.

Le test est répété plusieurs fois. Si l'interrogateur parvient à une conclusion correcte dans la moitié des tests ou moins, l'ordinateur est considéré comme ayant une intelligence artificielle, car l'interrogateur le considère comme un répondant humain.



Le Test de Turing : L'interrogateur, joueur C, doit déterminer lequel des joueurs, A ou B, est un ordinateur et lequel est un humain. Il peut seulement utiliser les réponses écrites aux questions pour faire sa décision.

@Pinar Saygin, Ayse, Ilyas Cicekli, and Varol Akman. "Turing test: 50 years later." Minds and machines 10.4 (2000): 463-518.

AI, ML, DL

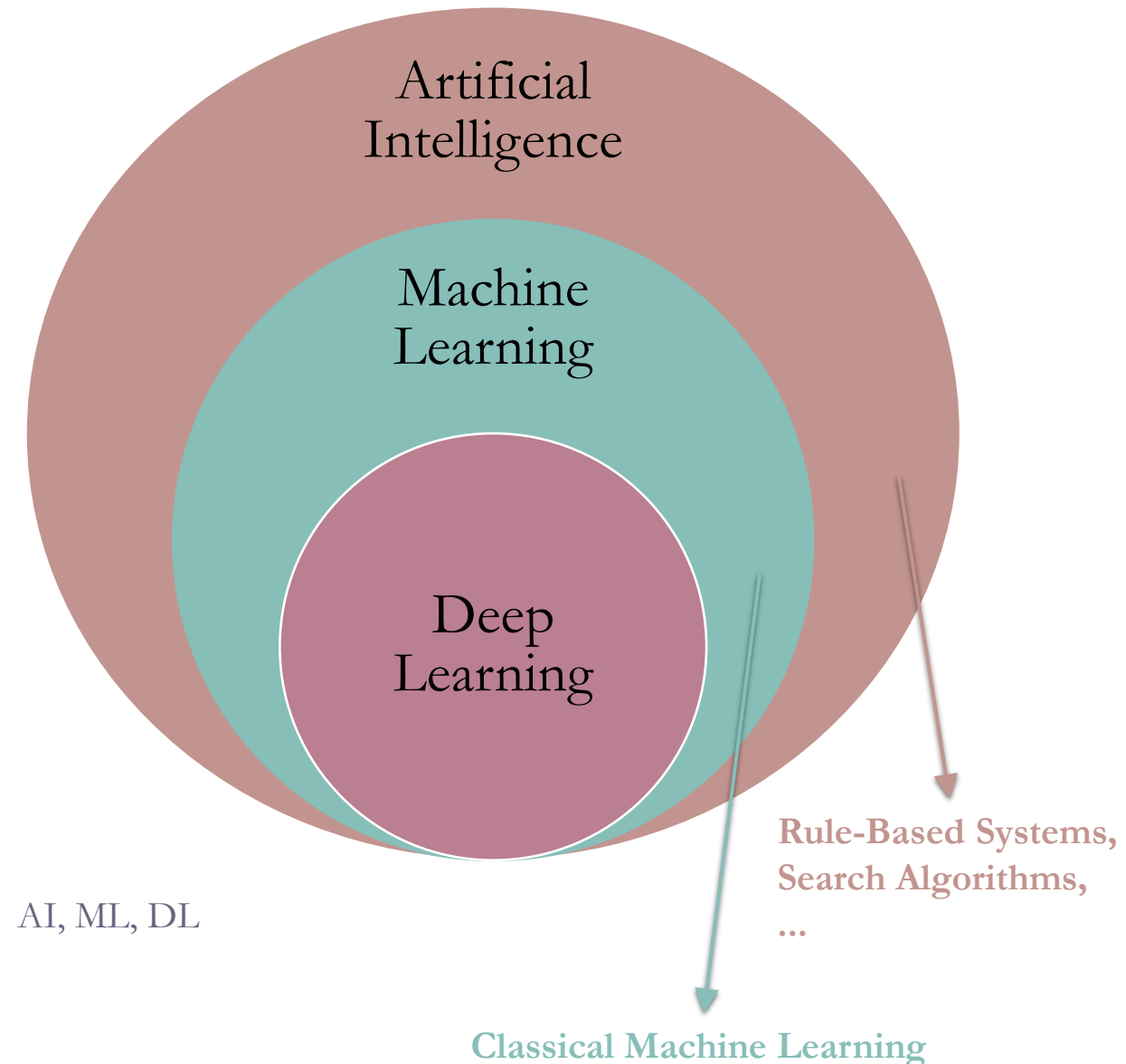
L'intelligence artificielle (IA) désigne l'utilisation de machines informatiques créées par l'homme, capables d'imiter certaines capacités humaines telles que la résolution de problèmes et la prise de décision.

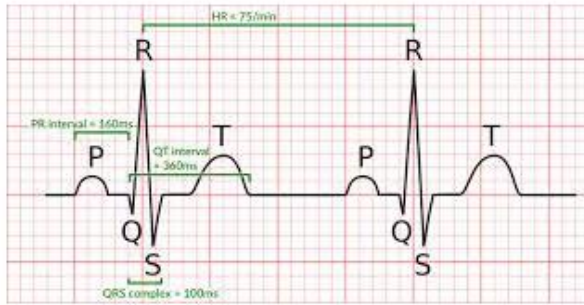
L'apprentissage automatique (ML) est une branche de l'IA. Il permet à un système informatique d'apprendre par lui-même et d'améliorer ses performances, sans être programmé explicitement pour chaque tâche (comme les systèmes basés sur des règles fixes).

Le ML se base sur l'analyse de données pertinentes à l'aide de modèles mathématiques. C'est grâce à cette analyse que le système est capable de trouver des relations et de faire des prédictions ou de prendre des décisions de manière autonome.

L'apprentissage profond (DL) est une sous-catégorie de l'apprentissage automatique qui utilise des réseaux neuronaux profonds (de milliers à des milliards de paramètres entraînaables) pour permettre aux machines de prendre des décisions complexes de manière autonome, en apprenant directement à partir de grandes quantités de données, sans intervention humaine.

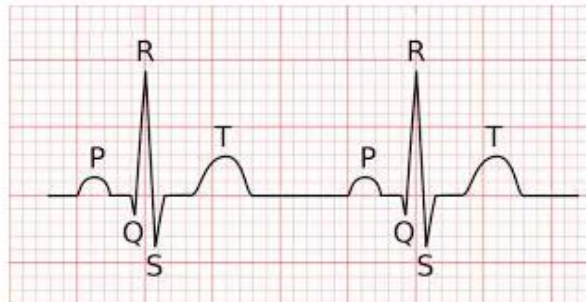
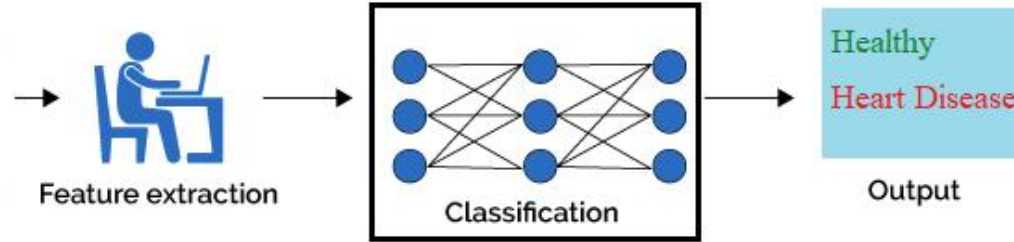
* Apprentissage automatique **classique** : repose généralement sur des caractéristiques définies par des experts.





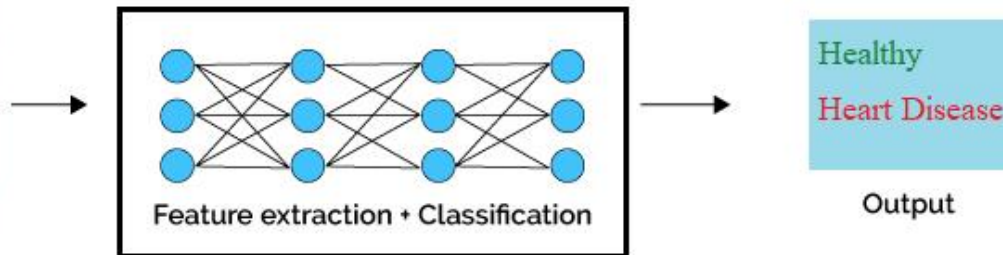
Input

Conventional Machine Learning



Input

Deep Learning



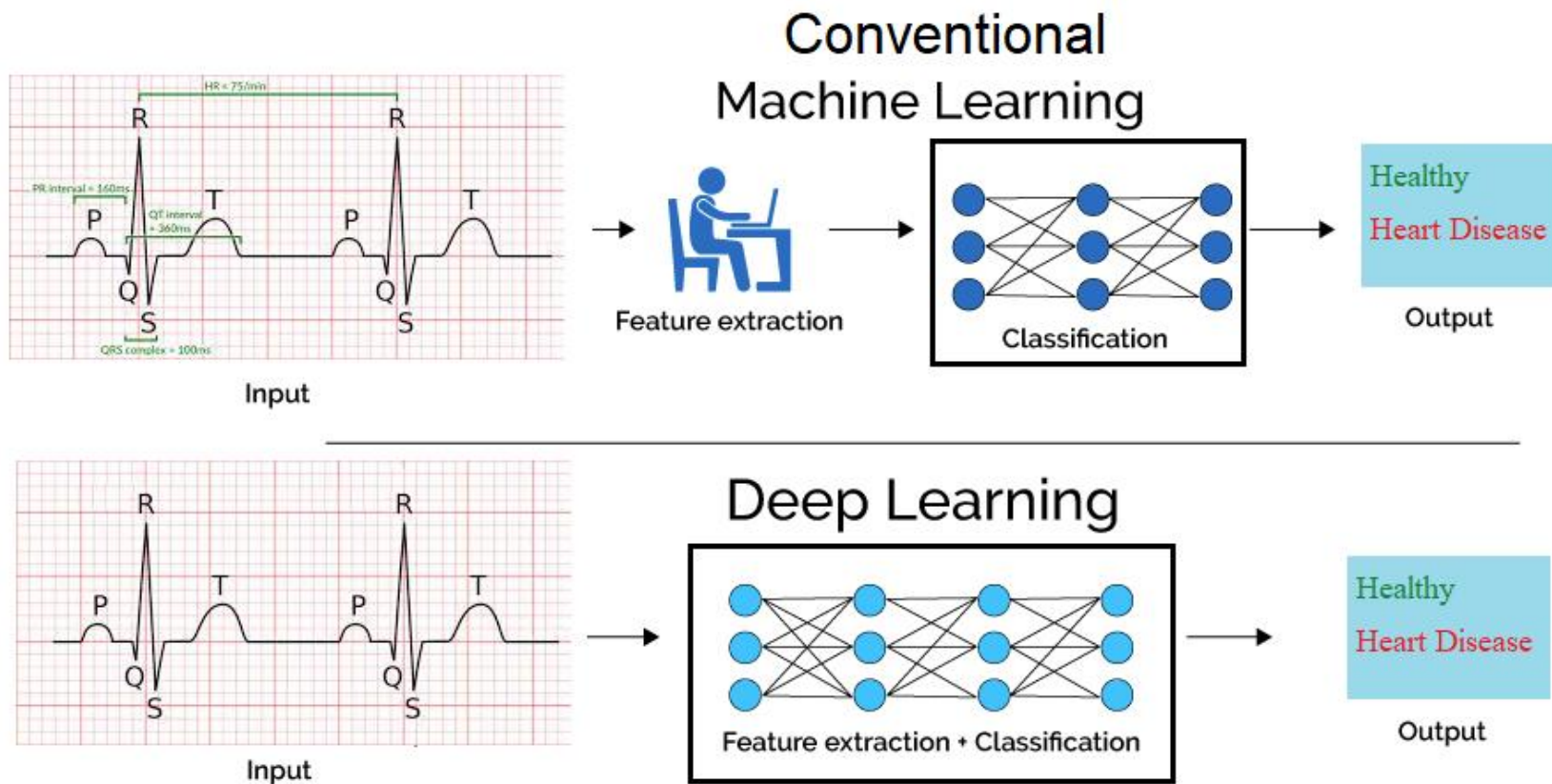
Exemple: Détection des maladies cardiaques à partir de l'ECG

ML classique : Lorsque les données sont limitées → les modèles profonds (avec beaucoup de paramètres) risquent de sur-apprendre (**Pourquoi ?**).

Les modèles profonds ont la capacité de mémoriser les données d'entraînement, y compris le bruit

On privilégie alors des modèles classiques (qui sont plus simples) (régression linéaire, SVM, forêts aléatoires) et on améliore leurs performances grâce à l'extraction de caractéristiques pertinentes (définies par un expert) (**Pourquoi ?**)

L'extraction de caractéristiques pertinentes simplifie le modèle et réduit le nombre de paramètres nécessaires, ce qui améliore la capacité d'apprentissage et la généralisation.

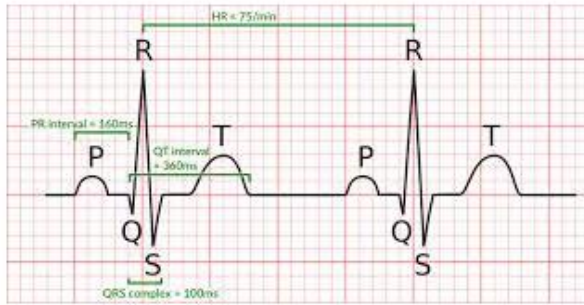


On peut également extraire des caractéristiques **définies par un expert** et les fournir à un modèle de deep learning. Cette approche peut simplifier l'apprentissage, mais elle est souvent évitée, car l'un des principaux avantages du deep learning est justement sa capacité à apprendre ces caractéristiques directement à partir des données brutes.

Exemple: Détection des maladies cardiaques à partir de l'ECG

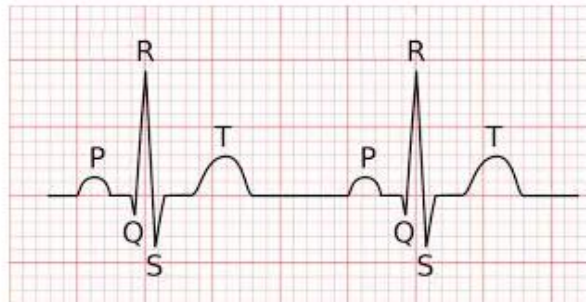
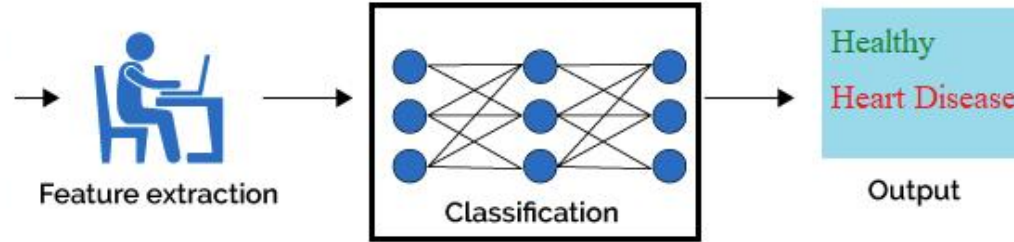
Deep Learning (DL) : Utilise des réseaux de neurones profonds (avec plusieurs couches et avec plusieurs paramètres) pour apprendre automatiquement.

- Avantage du DL : Peut traiter des signaux ECG bruts sans prétraitement.
- Limite du DL : Plus de paramètres → besoin de plus de données.



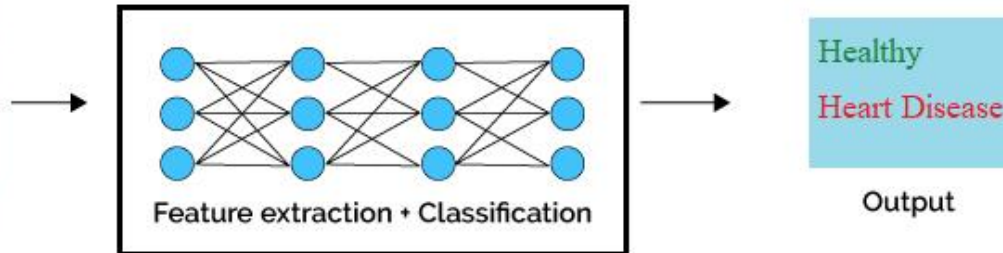
Input

Conventional Machine Learning



Input

Deep Learning



Exemple: Détection des maladies cardiaques à partir de l'ECG

En général, le nombre de paramètres dans les modèles ML classiques est souvent directement lié au **nombre de caractéristiques d'entrée** (relation directe) et à **la taille de l'ensemble de données** (moins il y a d'exemples d'apprentissage, plus il faut utiliser des modèles simples pour éviter le surajustement), et il se compte généralement en dizaines, centaines ou milliers.

Les modèles d'apprentissage profond, notamment les réseaux neuronaux, se caractérisent par leur grand nombre de paramètres entraînaables. Ces paramètres sont les poids et les biais des connexions entre les neurones dans différentes couches. Ils sont généralement de l'ordre de milliers à milliards.

Terminologie du ML

Données brutes : Données primaires collectées à partir d'une source, représentant un phénomène sans aucun traitement, transformation ou extraction de caractéristiques.

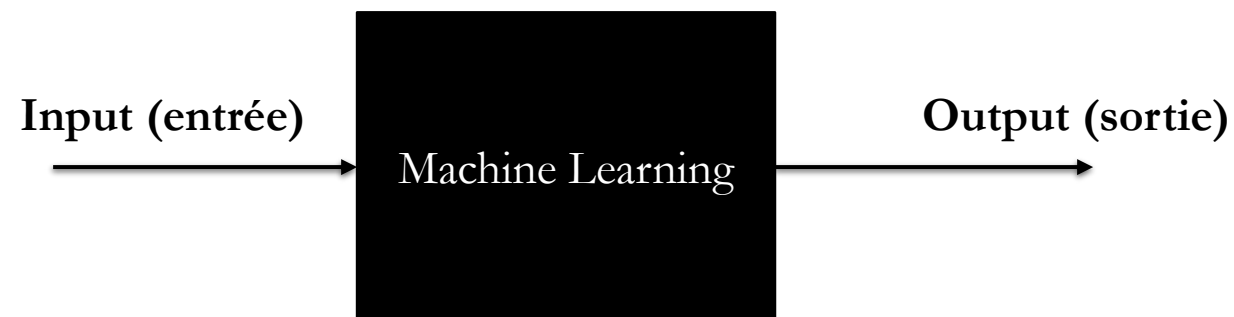
Exemple : Signal ECG brut.

Vecteur de caractéristiques (vecteur d'entrée) ($\mathbf{x}_i$) :

Représentation des données d'entrée sous forme d'un vecteur numérique de dimension d , prêt à être traité par un algorithme de ML. Selon le cas, les données brutes peuvent être utilisées directement comme vecteur de caractéristiques ou faire l'objet d'un prétraitement, d'une transformation ou d'une extraction de caractéristiques avant d'être converties en vecteur.

Exemple : Caractéristiques extraites d'un signal ECG représentant le timing et l'amplitude des événements électriques cardiaques.

Exemples d'entraînement (observations) ($\mathbf{X}_{d \times N}$) : Un modèle de ML doit voir de nombreux exemples d'entrées et de sorties pour apprendre leur relation. Chaque entrée est appelée un **exemple d'entraînement**.



Feature vector :

$$\mathbf{x}_{d \times 1} = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \\ \vdots \\ x(d) \end{bmatrix}$$

Matrice des exemples d'entraînement :

$$\mathbf{X}_{d \times N} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$$

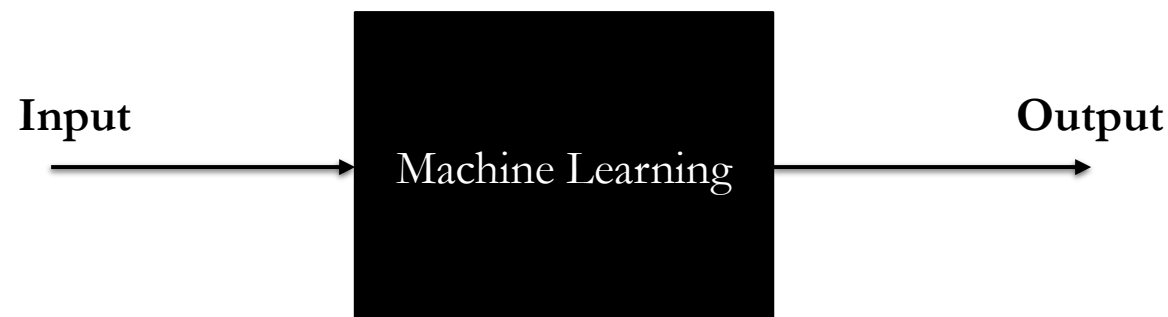
Terminologie du ML

Label (étiquette) (cible) (vérité terrain) (z_i): Un label est la valeur correcte ou la classe d'un exemple d'entraînement qui doit être prédite par le modèle de ML. Il permet au modèle d'apprendre à faire des prédictions lorsqu'il rencontre des exemples sans label.

Notre objectif est de minimiser la différence entre la sortie du modèle et le label, c'est-à-dire obtenir des prédictions aussi proches que possible des labels.

Sortie (y_i): Classe ou valeur prédite pour un exemple d'entraînement.

Remarque : Ces notations sont pour une sortie à une dimension.



Feature vector :

$$\mathbf{x}_{d \times 1} = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \\ \vdots \\ x(d) \end{bmatrix}$$

Matrice des exemples d'entraînement :

$$\mathbf{X}_{d \times N} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$$

Matrice des labels :

$$\mathbf{z}_{1 \times N} = [z_1, z_2, \dots, z_N]$$

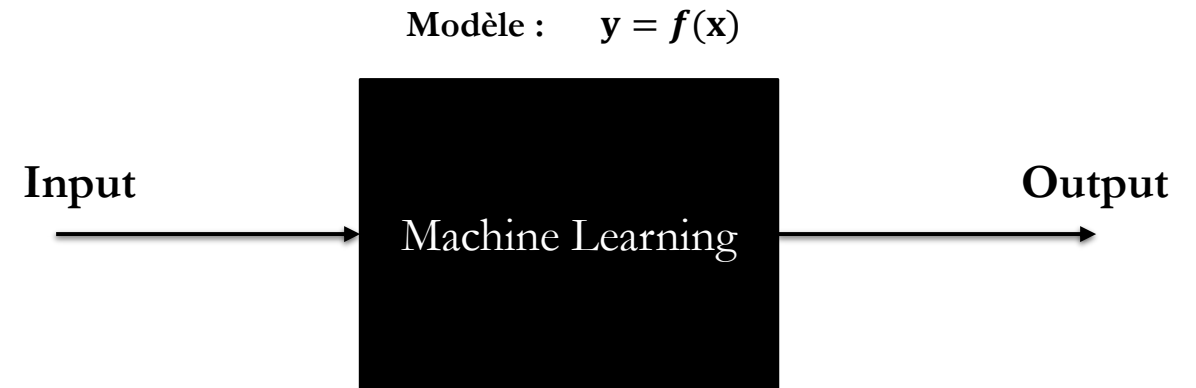
Matrice des sorties (prédites) :

$$\mathbf{y}_{1 \times N} = [y_1, y_2, \dots, y_N]$$

Terminologie du ML

Fonction de coût (perte) (objectif) $L(\mathbf{z}, \mathbf{y})$: Le critère mathématique qui quantifie la performance du modèle d'apprentissage automatique. C'est essentiellement une mesure de la différence entre la sortie prédite et le label.

Modèle $f(\cdot)$: La relation explicite (ou implicite) entrée-sortie qui doit être entraîné à l'aide de données d'entraînement.



Feature vector :

$$\mathbf{x}_{D \times 1} = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \\ \vdots \\ x(d) \end{bmatrix}$$

Matrice des exemples d'entraînement :

$$\mathbf{X}_{d \times N} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$$

Matrice des labels :

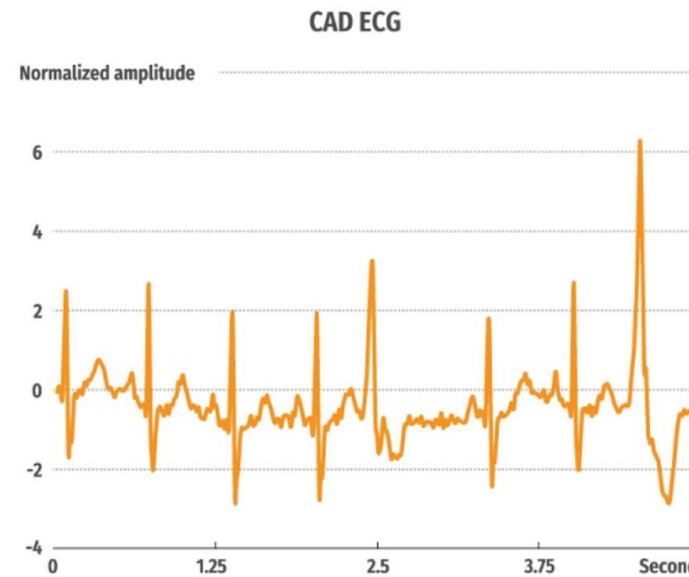
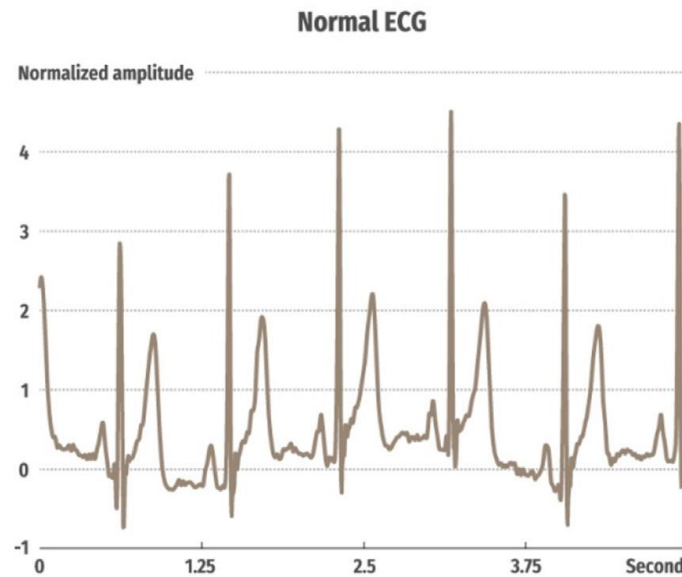
$$\mathbf{z}_{1 \times N} = [z_1, z_2, \dots, z_N]$$

Fonction de coût :

$$C(\mathbf{z}, \mathbf{y})$$

Matrice des sorties (prédites) :

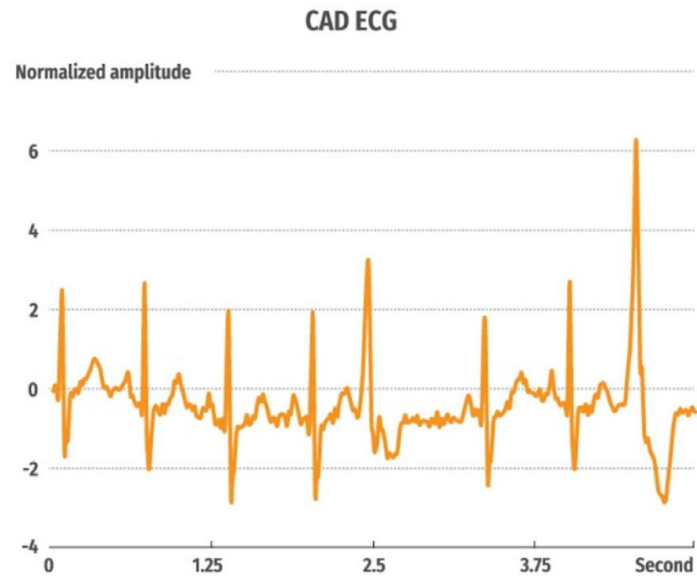
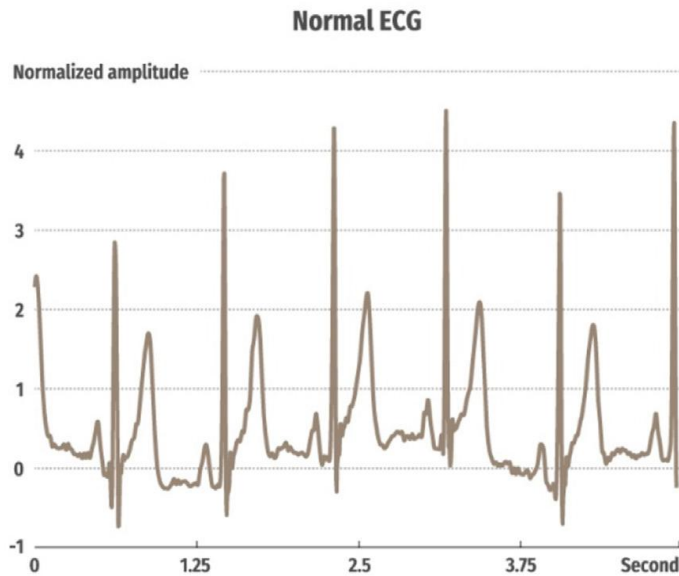
$$\mathbf{y}_{1 \times N} = [y_1, y_2, \dots, y_N]$$



Exemple

Détection du degré (en pourcentage) de maladie coronarienne (CAD) dans un ensemble de données de 200 signaux ECG (dérivation II).

- La maladie coronarienne se caractérise par l'obstruction ou le rétrécissement des vaisseaux sanguins alimentant le cœur, ce qui peut provoquer des douleurs thoraciques ou une crise cardiaque..
- La maladie coronarienne est causée par l'accumulation de plaque (dépôts de cholestérol) dans la paroi des artères qui irriguent le cœur (appelées artères coronaires). Ce processus est appelé **athérosclérose**.
- La maladie coronarienne peut changer le tracé de l'ECG :
 - Dépression/élévation du segment ST
 - Aplatissement des ondes T
 - Ondes T inversées
 - Ondes Q anormales



Exercice : Nous voulons développer un algorithme de ML pour la détection du degré de la CAD en pourcentage à partir du signal ECG

- 1- Décrivez quel pourrait être un vecteur d'entrée/feature $\mathbf{x}$,
- 2- Quelle est la taille de la matrice des exemples d'entraînement $\mathbf{X}$,
- 3- Quels seraient les labels $\mathbf{z}$ possibles (Comment définir et obtenir les labels ?) et
- 4- Proposez une fonction de cout qui peut caractériser la distance entre les estimations d'un algorithme d'apprentissage automatique et les labels.

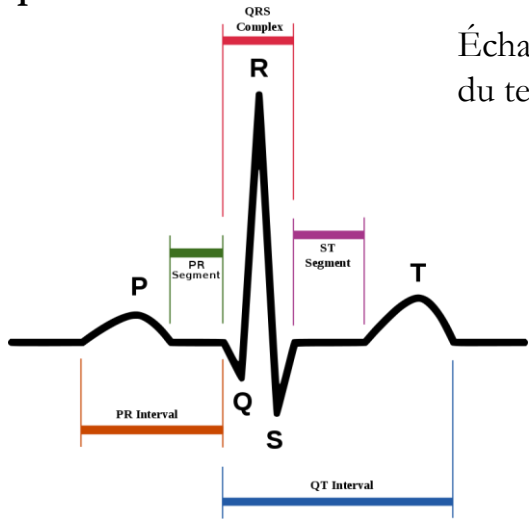
Exemple

Veillez envoyer la réponse par courriel :
mohamad.forouzanfar@etsmtl.ca.

Objet du courriel : « GTS880: Cours 3- Exercice 1 »

N'oubliez pas vos noms

Raw input:



Feature vector 1 :

Échantillons bruts d'ECG au fil du temps (intervalle d'échantillonnage = T sec):

$$\mathbf{x}_{d \times 1} = \begin{bmatrix} ecg(0) \\ ecg(T) \\ ecg(2T) \\ \vdots \\ ecg(end) \end{bmatrix}$$

Feature vector 2:

Caractéristiques morphologiques extraites :

$$\mathbf{x}_{d \times 1} = \begin{bmatrix} PR_{duration} \\ QRS_{duration} \\ ST_{duration} \\ QT_{duration} \\ P_{amplitude} \\ R_{amplitude} \\ T_{amplitude} \\ \vdots \\ \vdots \\ \vdots \end{bmatrix}$$

Exemples de données (N=200) : Le nombre de sujets dans notre ensemble de données.

$$\mathbf{X}_{d \times N} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$$

labels : 0 %–100 %, en utilisant un standard de référence tel que l'angiographie coronarienne par tomodensitométrie (TDM)(CT coronary angiogram).

$$\mathbf{z}_{1 \times N} = [5\%, 20\%, 0\%, 80\%, \dots]$$

Une autre méthode pour obtenir des labels consiste à utiliser une procédure traditionnelle basée sur un tube flexible (cathéter) introduit par l'aine ou le bras jusqu'aux artères coronaires.

Sorties prédites possibles : $\mathbf{y}_{1 \times N} = [7\%, 28\%, 10\%, 65\%, \dots]$

Fonction de coût :

$$MSE = \frac{1}{N} \sum_{i=1}^N (z_i - y_i)^2$$

Example

Detection of the degree (in percentage) of coronary artery disease (CAD) in a dataset of 200 ECG signals (lead II)

- Coronary artery disease (CAD) is when the blood vessels supplying the heart become blocked or narrowed and this can cause chest pain or a heart attack.
- CAD is caused by plaque (cholesterol deposits) buildup in the wall of the arteries that supply blood to the heart (called coronary arteries). This process is called **atherosclerosis**
- A specialist can detect CAD from ECG pattern:
 - ST segment depression/elevation
 - Flattening of the T waves
 - Inverted T waves
 - Abnormal Q waves

La régression linéaire :

la technique d'IA la plus simple

L'un des problèmes de base en statistiques est de trouver une relation linéaire entre certaines variables indépendantes (caractéristiques d'entrée) et un résultat spécifique (label). Cela s'appelle la régression linéaire.

Supposons que nous ayons un ensemble d'observations $\{\mathbf{x}_i\}_{i=1}^N$, le i -ème sortie prédite est modéliser (relation entrée-sortie) comme suit :

$$y_i = \alpha_0 + \alpha_1 x_i(1) + \alpha_2 x_i(2) + \dots + \alpha_d x_i(d)$$

Ce qui peut être reformulé sous forme vectorielle :

$$y_i = \boldsymbol{\alpha}^T \mathbf{x}_i$$

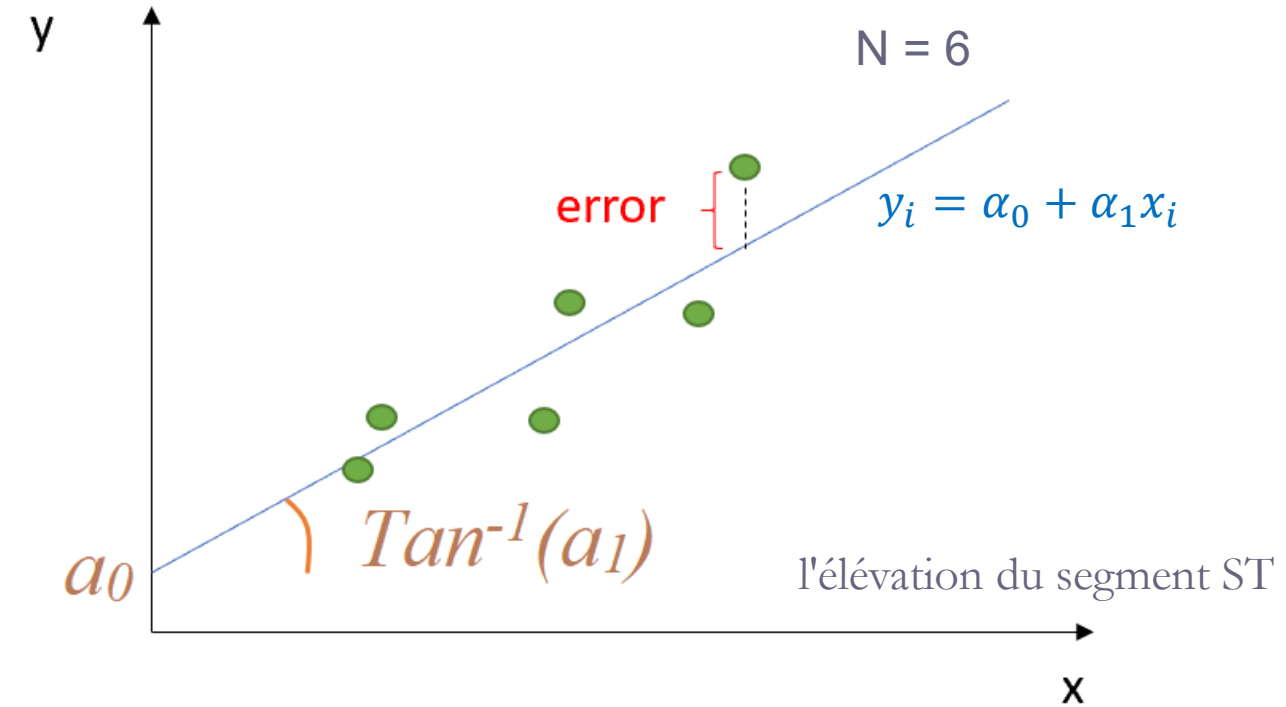
Où $\mathbf{x}_i = [1, x_i(1), x_i(2), \dots, x_i(d)]^T$ est le vecteur caractéristique pour l'exemple i , et $\boldsymbol{\alpha} = [\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_d]^T$ est le vecteur des paramètres inconnus (à estimer).

Remarque : La transposée T d'une matrice est obtenue en échangeant ses lignes et colonnes, ou ses colonnes et ligne.

α_0 inclut des influences non observées sur y_i provenant de sources autres que x_i .

➔ Nous avons N équations (mesures) (exemples) (observations) et $d+1$ inconnues (paramètres).

degré de la CAD



Exemple : Relation entre l'élévation du segment ST' (x) et le degré de la maladie coronarienne (CAD) (y) (données synthétiques).

$N = 6, d + 1 = 2$

Types de systèmes en régression linéaire

1. Système sous-déterminé (sous-contraint) :

Nombre d'équations < nombre d'inconnues

→ Il existe généralement un nombre infini de solutions pour

2. Système carré :

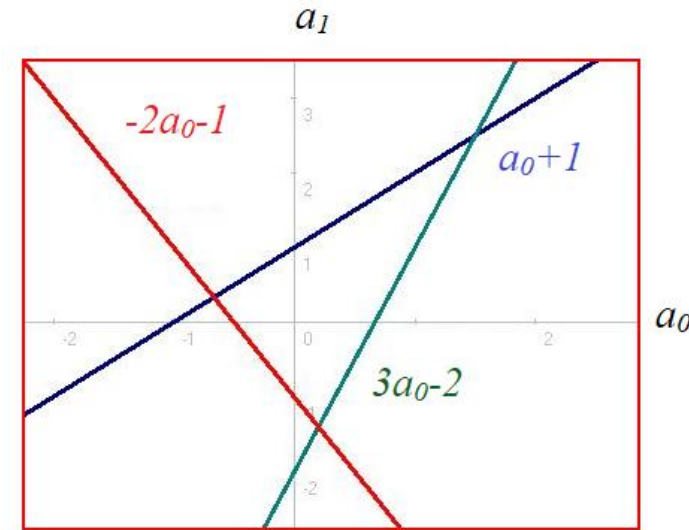
Nombre d'équations = nombre d'inconnues

→ Il existe généralement une solution unique

3. Système surdéterminé (surcontraint) :

Nombre d'équations > nombre d'inconnues

→ Il n'existe généralement pas de solution exacte



Exemple:

1. Système sous-déterminé :

$$\{\alpha_1 = -2\alpha_0 - 1$$

→ De nombreuses solutions

2. Système équilibré :

$$\begin{cases} \alpha_1 = -2\alpha_0 - 1 \\ \alpha_1 = 3\alpha_0 - 2 \end{cases}$$

→ Une solution unique

3. Système surdéterminé :

$$\begin{cases} \alpha_1 = -2\alpha_0 - 1 \\ \alpha_1 = 3\alpha_0 - 2 \\ \alpha_1 = \alpha_0 + 1 \end{cases}$$

→ Pas de solution exacte

Où en sommes-nous en machine learning ?

→ **Système sous-déterminé** : Se produit en apprentissage automatique avec des modèles complexes (ex. réseaux profonds). Il existe alors une infinité de solutions qui s'ajustent parfaitement aux données d'entraînement, ce qui peut mener au surapprentissage et à de mauvaises performances sur de nouvelles données..

→ **Système carré** : Avec le bruit et les erreurs de mesure, avoir autant d'équations que d'inconnues ne suffit pas et le système se comporte plutôt comme un système sous-déterminé.

→ **Système surdéterminé** : Cas fréquent en ML (ex. régression linéaire) où il y a plus de données que de paramètres. Aucune solution ne satisfait tous les points, on cherche donc la meilleure approximation en minimisant une fonction de coût.

→ En IA, le scénario idéal est généralement un système surdéterminé où vous disposez d'une grande quantité de données de haute qualité. Mais le risque de surajustement existe toujours.

Moindres Carrés Ordinaires (OLS) :

La solution classique d'un modèle linéaire surdéterminé

Les mathématiques sont facultatives

OLS est une méthode statistique pour estimer les paramètres inconnus dans un modèle de régression linéaire surdéterminé :

$$\mathbf{y} = \boldsymbol{\alpha}^T \mathbf{X}$$

(("T" est l'opérateur de transposition : un opérateur qui inverse une matrice sur sa diagonale, c'est-à-dire qu'il échange les lignes et les colonnes.))

$$\mathbf{X} = \begin{bmatrix} 1 & \cdots & 1 \\ x_1(1) & \ddots & x_N(1) \\ \vdots & \ddots & \vdots \\ x_1(d) & \cdots & x_N(d) \end{bmatrix}, \quad \boldsymbol{\alpha} = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_d \end{bmatrix},$$

$$\mathbf{y} = [y_1, \dots, y_N], \mathbf{z} = [z_1, \dots, z_N]$$

Un tel système n'a généralement pas de solution exacte, donc l'objectif est plutôt de trouver les paramètres α_i qui "mieux" ajustent les équations, en résolvant un problème de minimisation :

$$\hat{\boldsymbol{\alpha}} = \underbrace{\arg \min}_{\boldsymbol{\alpha}} \mathcal{C}(\boldsymbol{\alpha})$$

(("argmin" est l'argument du minimum : l'ensemble des valeurs $\boldsymbol{\alpha}$ pour lesquelles la fonction $\mathcal{C}$ atteint un minimum.))

$$\mathcal{C}(\boldsymbol{\alpha}) = \sum_{i=1}^N (z_i - y_i)^2$$

Pour trouver $\boldsymbol{\alpha}$:

$$\frac{\partial \mathcal{C}(\boldsymbol{\alpha})}{\partial \boldsymbol{\alpha}} = 0$$

En faisant un peu d'algèbre linéaire, on obtient :

$$\hat{\boldsymbol{\alpha}} = (\mathbf{X}\mathbf{X}^T)^{-1} \mathbf{X} \mathbf{z}^T$$

*Pour ceux qui sont intéressés, preuve des OLS :

https://web.stanford.edu/~mrosenfe/soc_meth_proj3/matrix_OLS_NYU_notes.pdf

Le calcul de l'inverse de $(\mathbf{X}^T \mathbf{X})$ est trop coûteux pour les grands ensembles de données. De plus, OLS ne s'applique pas aux modèles non linéaires.

Descente de Gradient (GD):

Une Approche Itérative pour l'Optimisation

La descente de gradient est un algorithme d'optimisation itératif de premier ordre utilisé pour trouver un minimum local d'une fonction différentiable. C'est l'un des éléments fondamentaux et très importants de ML/DL pour minimiser une fonction de coût.

Considérons une fonction de coût différentiable multi-variable $C(\alpha)$ dans un voisinage d'un point α^0 . Dans ce voisinage, $C(\alpha)$ diminue le plus rapidement lorsqu'on part du point α^0 dans la direction du gradient négatif de C en α^0 , c'est-à-dire, $-\mathbf{g}_C(\alpha^0) = -\nabla C(\alpha^0) = -\frac{\partial C(\alpha^0)}{\partial \alpha^0}$.

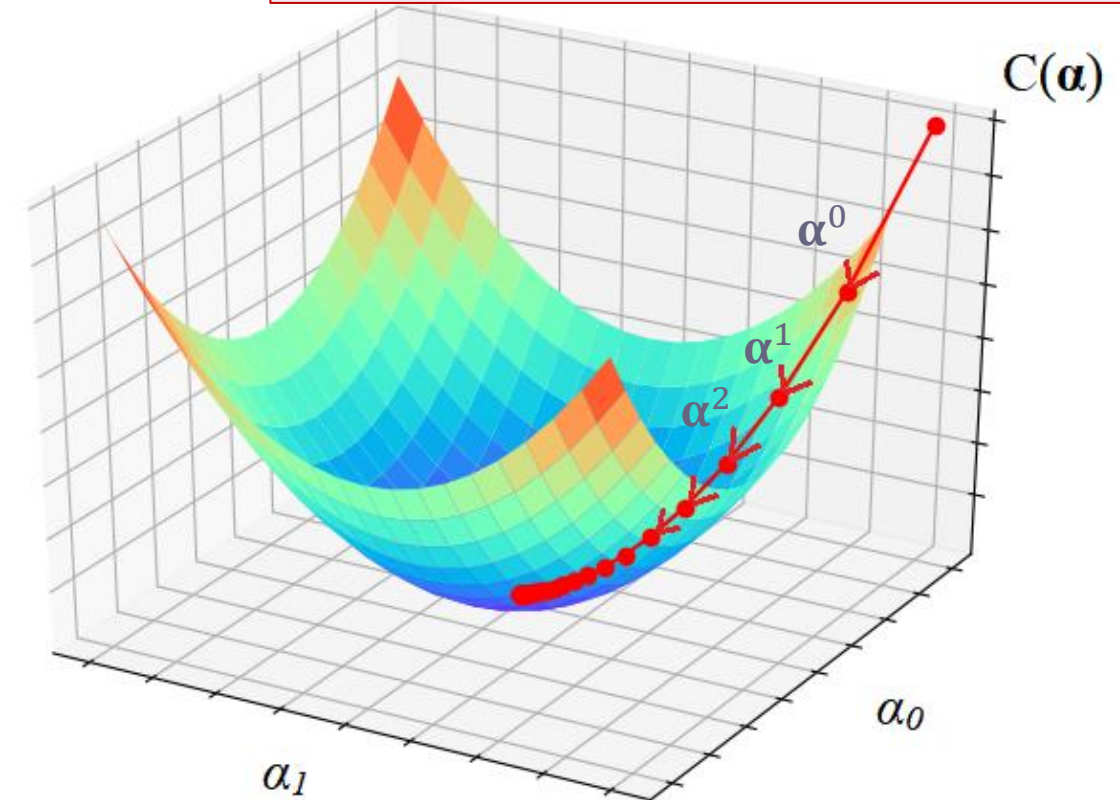
Par conséquent, en soustrayant une petite quantité (puisque nous n'avons pas encore beaucoup d'informations sur le comportement en dehors de ce voisinage) du gradient du point actuel α^0 , nous pouvons nous déplacer vers un point où la fonction de coût est plus faible :

$$\alpha^{n+1} = \alpha^n - \eta \mathbf{g}_C(\alpha^n) \\ C(\alpha^0) \geq C(\alpha^1) \geq C(\alpha^2) \dots$$

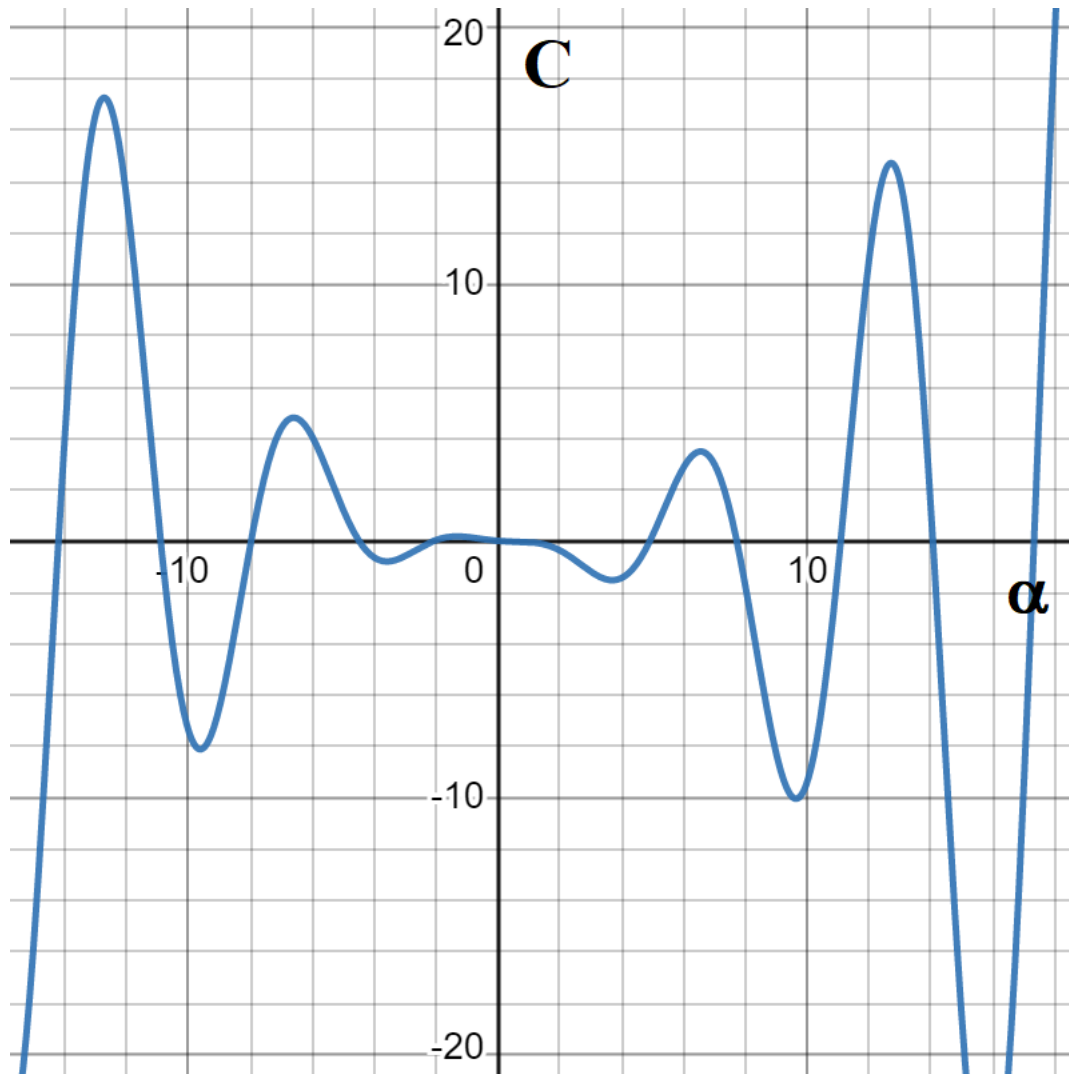
où $\eta \in \mathbb{R}_+$ est un petit pas, également appelé taux d'apprentissage, qui détermine la quantité de gradient à soustraire. « n » représente le nombre d'itérations.

$$\alpha = \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix}$$

Remarque : l'exposant indique le nombre d'itérations d'apprentissage. Par exemple, α^0 est le vecteur de paramètres initial.



La descente de gradient est une approche itérative où, à chaque itération, nous faisons un petit pas vers le minimum/maximum de la fonction de coût. Ce pas est contrôlé par le taux d'apprentissage. La raison pour laquelle nous choisissons de petits pas vers le maximum/minimum est que nos approximations sont basées sur les statistiques locales de notre point actuel.

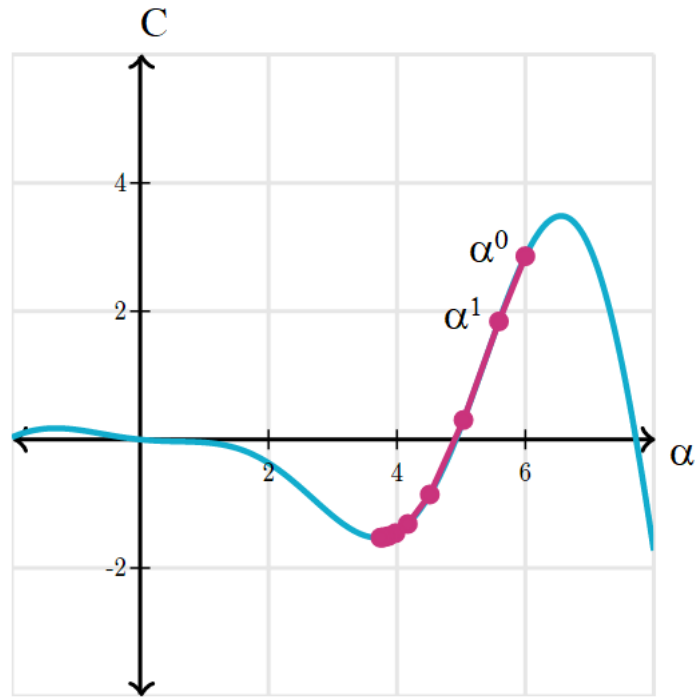


Exemple

Fonction de coût 1D :

$$C(\alpha) = \frac{\alpha^2 \cos(\alpha)}{10} - \frac{\alpha}{10}$$

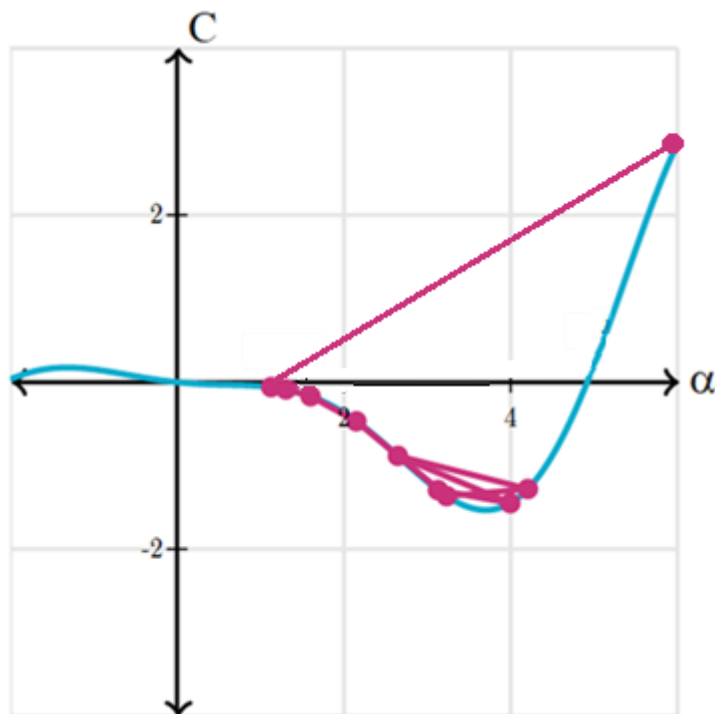
- Le point initial de départ de la recherche est important.
- La descente de gradient ne peut pas dire si un minimum qu'elle a trouvé est local ou global.
- Le taux d'apprentissage détermine si l'algorithme converge vers un minimum rapidement ou lentement, ou s'il diverge.



Exemple

$$C(\alpha) = \frac{\alpha^2 \cos(\alpha)}{10} - \frac{\alpha}{10}$$

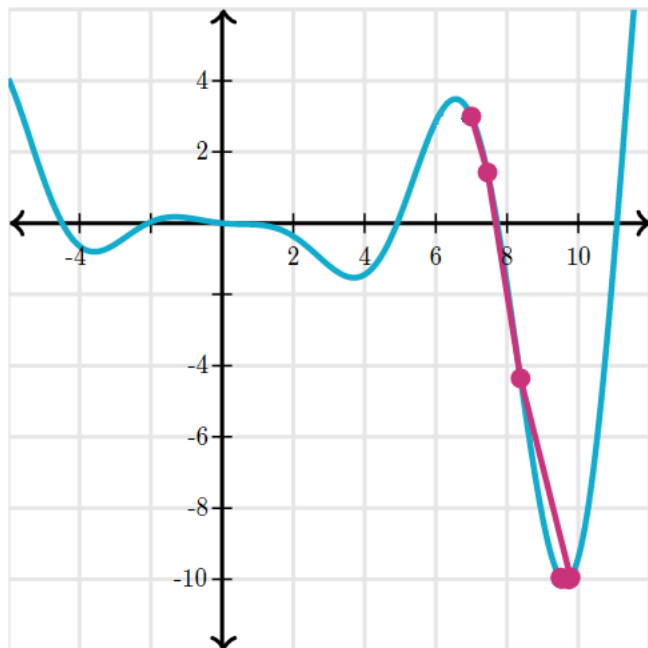
- En choisissant $\alpha^0 = 6$ et $\eta = 0.2$, après plusieurs itérations de l'algorithme GD, il converge vers un minimum local.



Exemple

$$C(\alpha) = \frac{\alpha^2 \cos(\alpha)}{10} - \frac{\alpha}{10}$$

- En choisissant $\alpha^0 = 6$ et $\eta = 1.5$, le GD ne converge pas vers un minimum local.
- **Question :** quel est l'impact du taux d'apprentissage sur la convergence ?



Exemple

$$C(\alpha) = \frac{\alpha^2 \cos(\alpha)}{10} - \frac{\alpha}{10}$$

- En choisissant $\alpha^0 = 7$ et $\eta = 0.2$, le GD va converger vers un autre minimum local.

Optimisation Avancée

Amélioration de Descente de Gradient

Descente de Gradient avec Momentum :

$$\alpha^{n+1} = \alpha^n - \eta(\rho \mathbf{v}^n + \mathbf{g}_C)$$

où $\mathbf{v}^n$ est la somme pondérée des derniers gradients et $\mathbf{g}_C$ est le gradient actuel. ρ contrôle l'impact des gradients précédents. Sa valeur typique est 0,9.

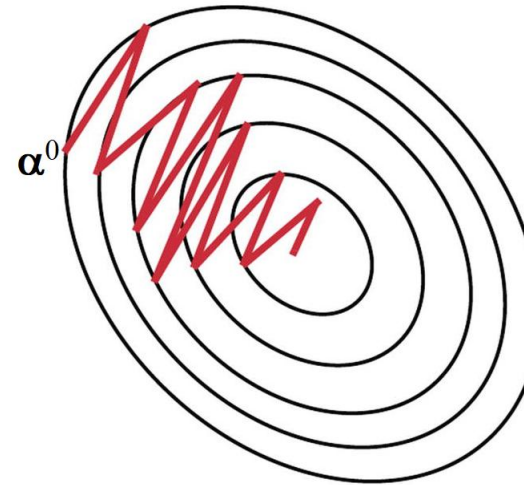
→ Cette méthode intègre une fraction des gradients précédents. Cela crée une « mémoire » des directions passées, ce qui permet d'accélérer la convergence vers le minimum et de réduire les oscillations.

RMSprop (root mean square propagation) :

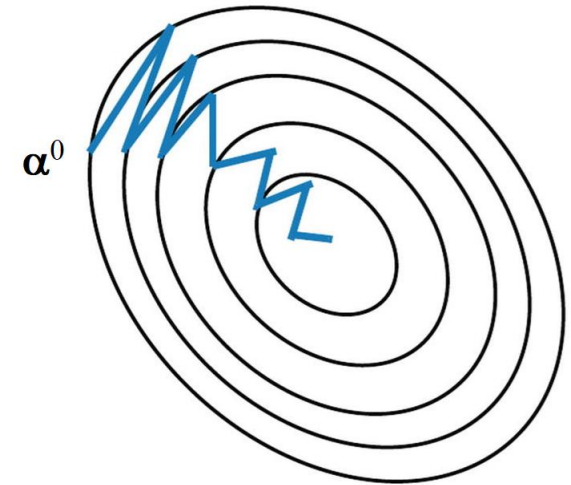
$$\alpha^{n+1} = \alpha^n - \frac{\eta}{\sqrt{s^{n+1}} + \varepsilon} \mathbf{g}_C$$

où s^{n+1} est la moyenne mobile pondérée des derniers gradients au carré. ε est une petite constante ajoutée pour éviter la division par 0.

→ L'idée principale est d'adapter le taux d'apprentissage en se basant sur une moyenne mobile des carrés des gradients passés. Si les gradients sont grands, le taux d'apprentissage diminue pour éviter les oscillations. S'ils sont petits, il augmente pour accélérer la convergence.



Gradient Descent **without** Momentum



Gradient Descent **with** Momentum

Optimisation Avancée

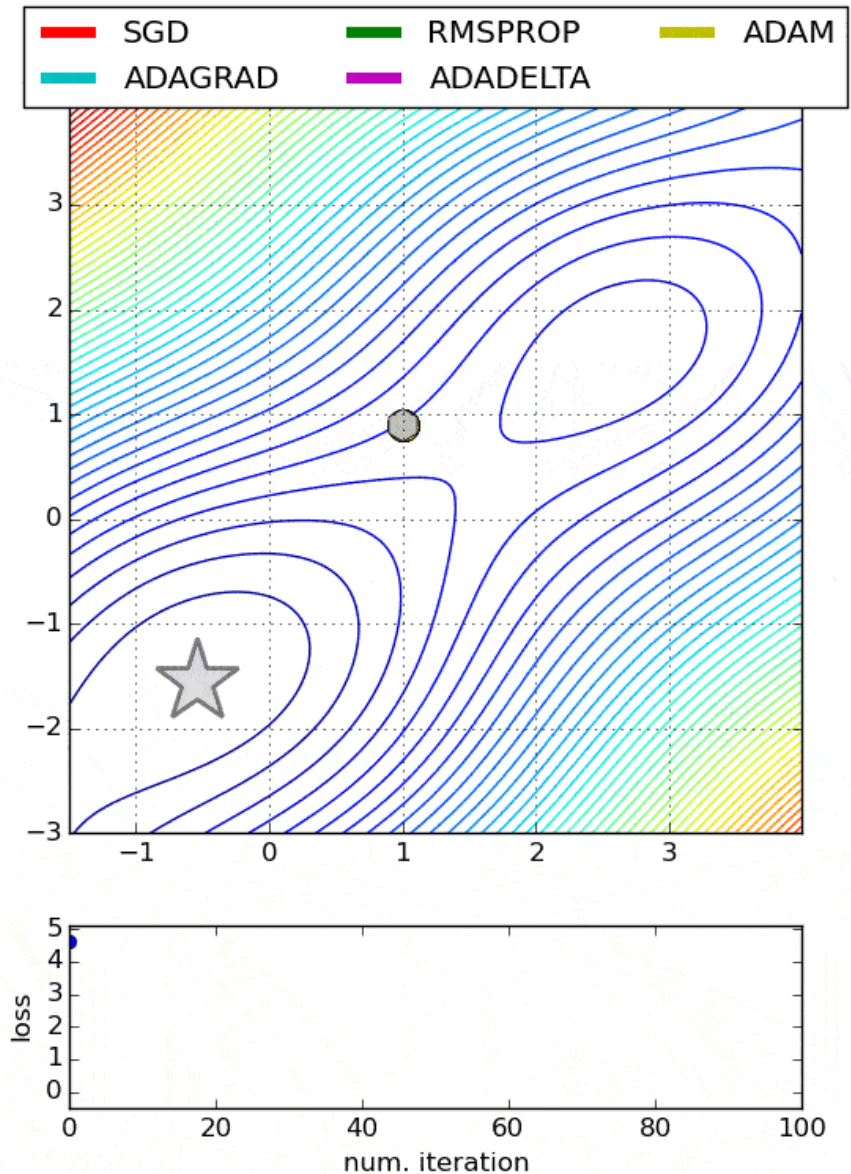
Amélioration de Descente de Gradient

Adam (Adaptive Moment Estimation) :

$$\alpha^{n+1} = \alpha^n - \frac{\eta}{\sqrt{s^{n+1}} + \varepsilon} m^{n+1}$$

où m^{n+1} est la moyenne mobile pondérée des derniers gradients : $(\rho v^n + g_c)$.

→ Adam combine RMSprop et momentum en utilisant une moyenne mobile des gradients au carré pour ajuster dynamiquement le taux d'apprentissage, tout en intégrant une moyenne mobile des gradients pour accélérer la convergence et réduire les oscillations.



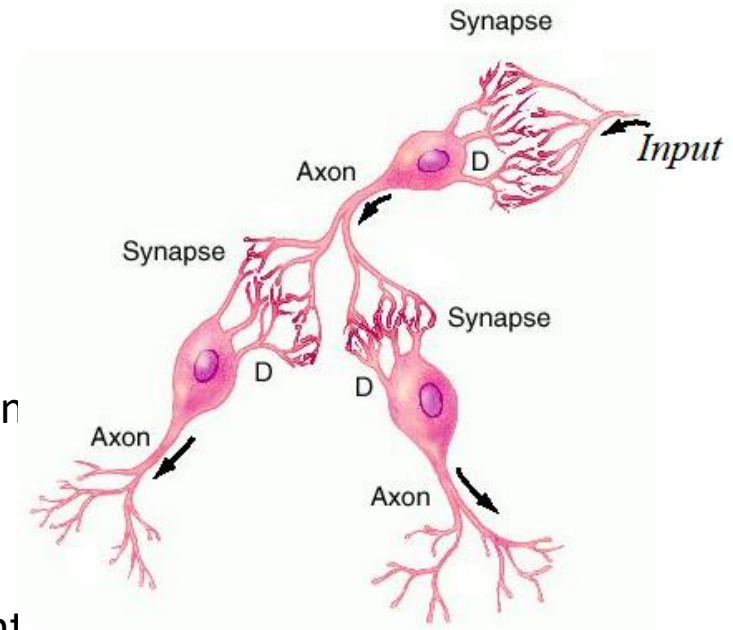
Réseaux neuronaux biologiques



<https://www.youtube.com/watch?v=oK3esXMQxaI>

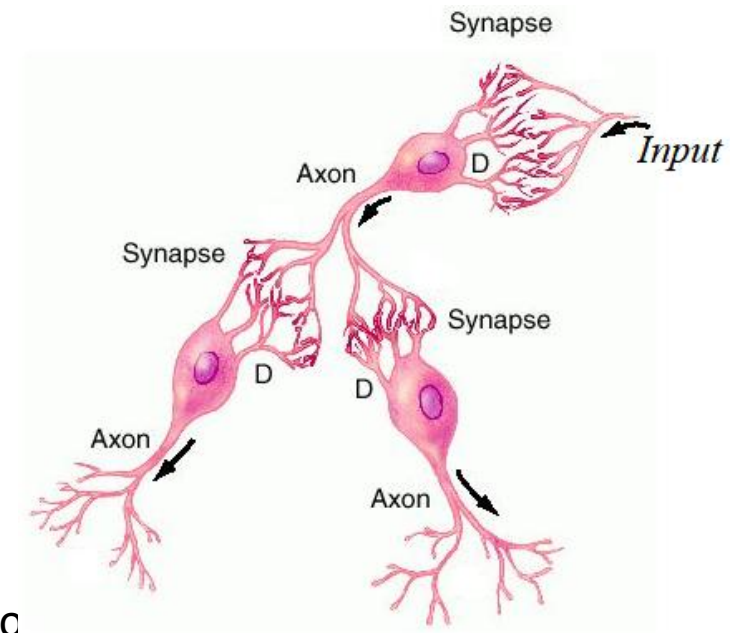
Inspiration - Neurone biologique - Résumé

1. **Dendrites:** Les dendrites sont les parties réceptrices d'un neurone, au début du neurone, qui reçoivent des signaux d'autres neurones. Les dendrites transmettent ces informations au corps cellulaire.
 -
2. **Soma (corps cellulaire) (noyau) :** C'est là que les signaux reçus des dendrites sont additionnés. Si la somme atteint un certain seuil, le neurone se déclenche.
 -



Inspiration - Neurone biologique - Résumé

3. **Axone** : Une longue fibre nerveuse qui sort du corps cellulaire et transporte le potentiel d'action (signal électrique) (l'information) du neurone vers d'autres neurones.
 -
4. **Synapse**: Jonction entre la terminaison de l'axone et la dendrite d'un autre neurone. Les neurotransmetteurs sont libérés à cet endroit pour propager le signal. La force du signal transmis est modulée par la force synaptique.
 -



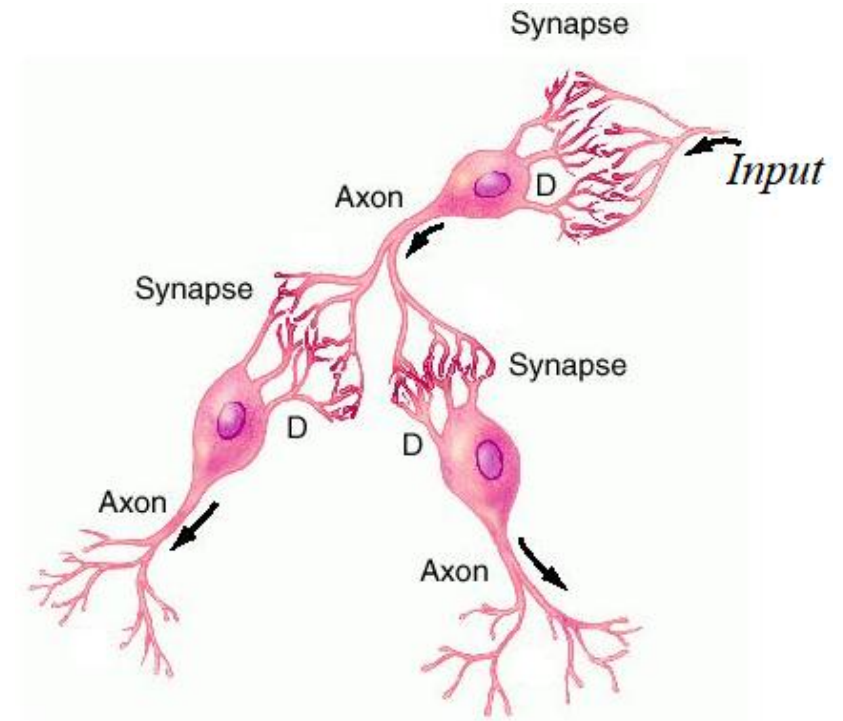
Inspiration - Neurone biologique

Discussion en groupe

Modèle de systèmes nerveux biologique:

Modélisez la fonction d'un neurone à l'aide d'opérateurs mathématiques. Utilisez un **schéma fonctionnel** montrant **les entrées, les sorties, la fonction noyau et la synapse**.

- Dendrites $\Rightarrow$ entrées du neurone
- Noyau $\Rightarrow$ unité de calcul (somme, seuil)
- Axone $\Rightarrow$ signal de sortie
- Synapse $\Rightarrow$ poids d'une entrée

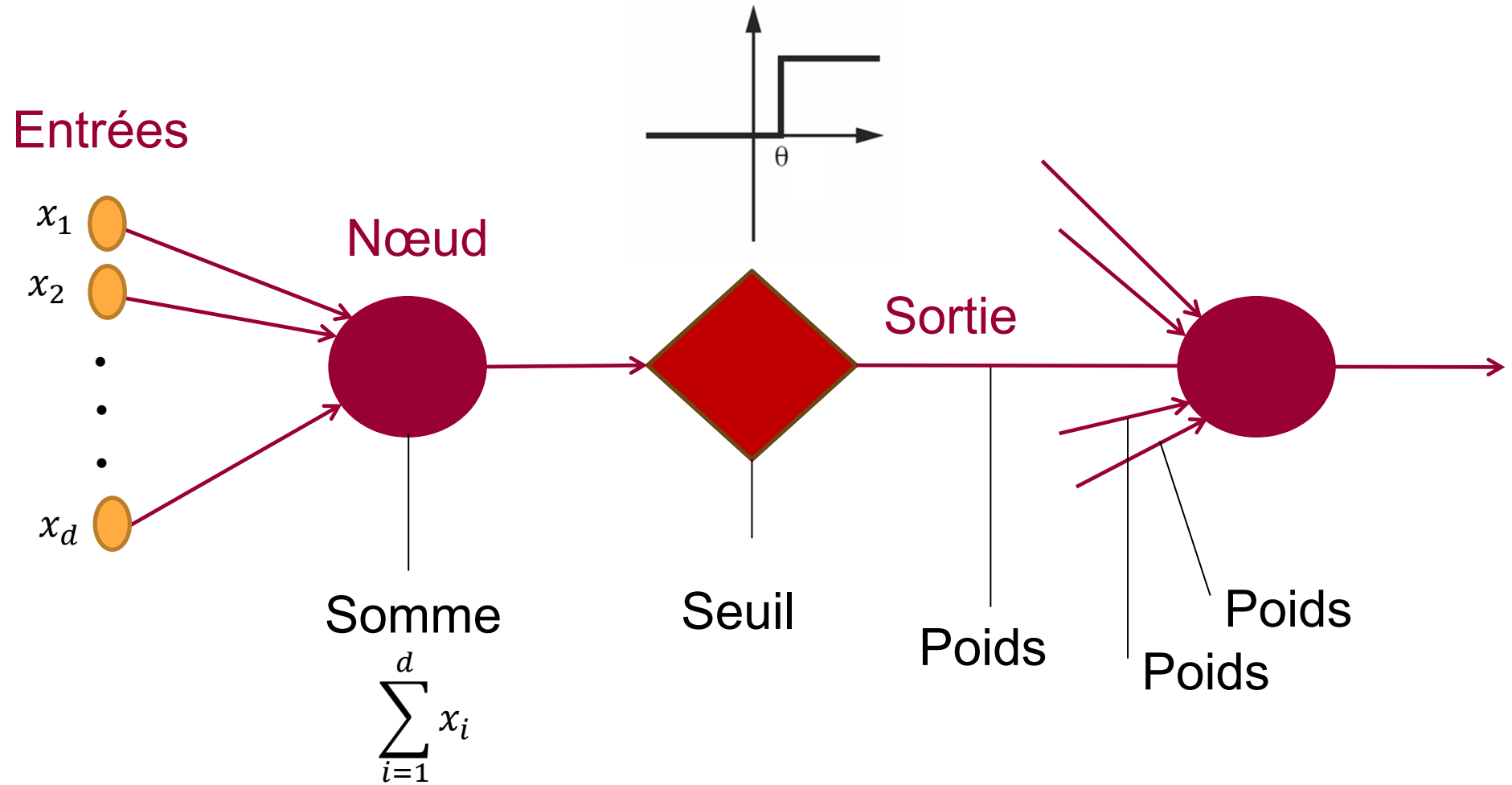


Veillez envoyer la réponse par courriel :
mohamad.forouzanfar@etsmtl.ca.

**Objet du courriel : « GTS880: Cours 3-
Exercice 2 »**

N'oubliez pas vos noms

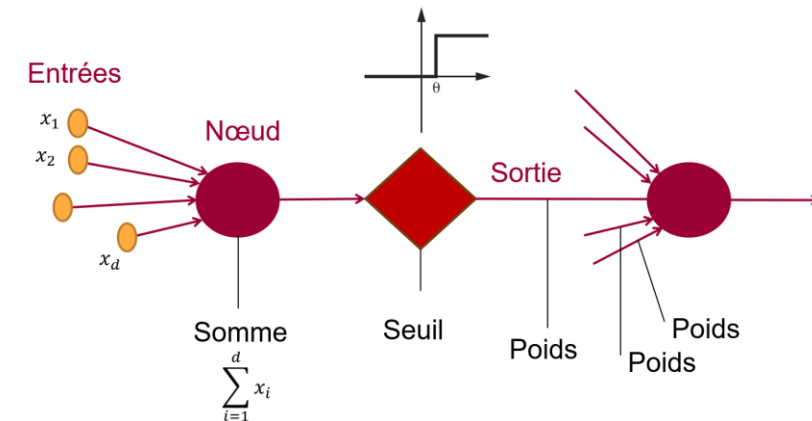
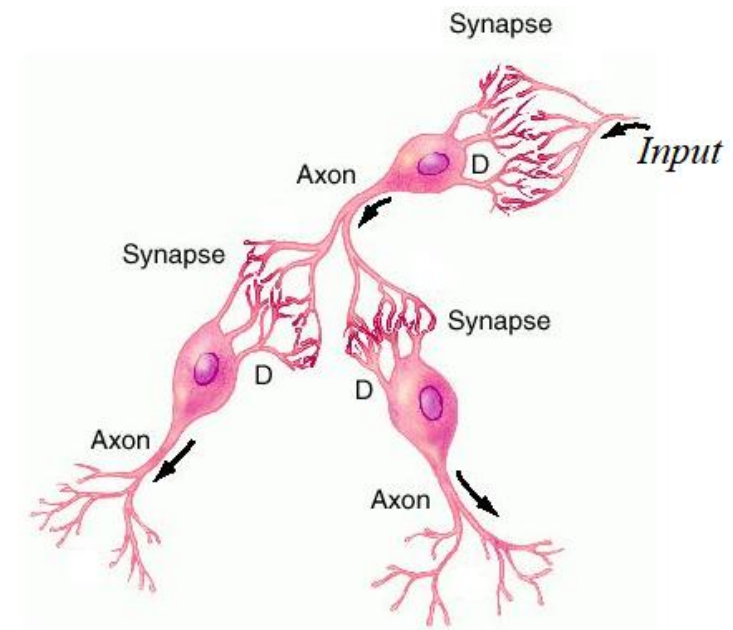
Inspiration - Neurone biologique



Inspiration - Neurone biologique - Résumé

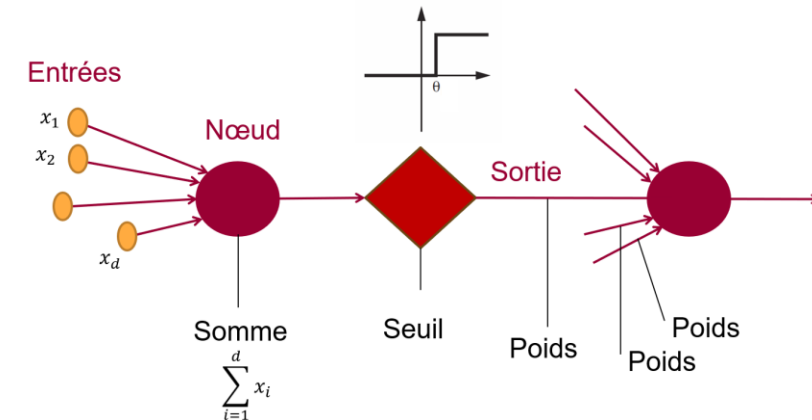
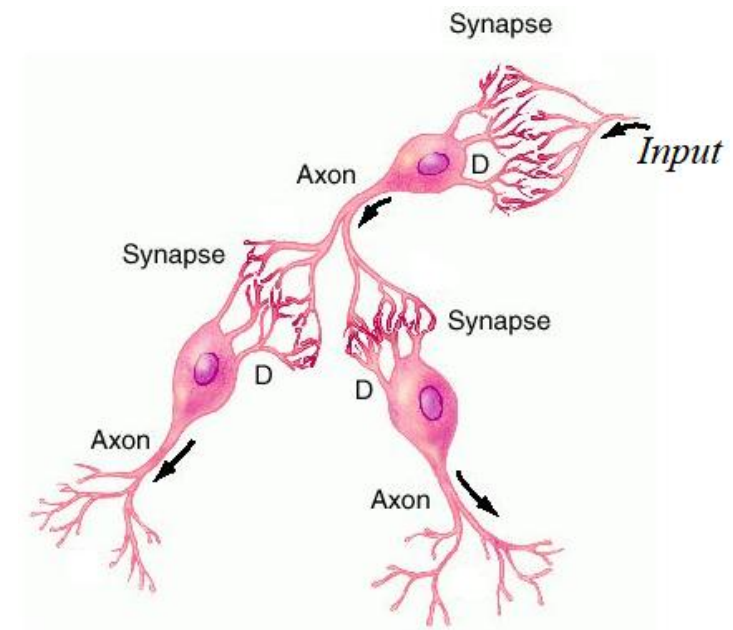
- 1. Dendrites:** Les dendrites sont les parties réceptrices d'un neurone, au début du neurone, qui reçoivent des signaux d'autres neurones. Les dendrites transmettent ces informations au corps cellulaire.
 - **Contrepartie artificielle :** La couche d'entrée ou les nœuds d'entrée d'un neurone artificiel, où chaque nœud correspond à une caractéristique spécifique de l'ensemble de données.
- 2. Soma (corps cellulaire) (noyau) :** C'est là que les signaux reçus des dendrites sont additionnés. Si la somme atteint un certain seuil, le neurone se déclenche.
 - **Contrepartie artificielle :** L'opération de somme pondérée dans un neurone artificiel, souvent combinée avec un terme de seuil (biais).
 - Le seuil est modélisé à l'aide d'une fonction seuil (échelon) (step function). Nous appelons cela une **fonction d'activation**.

Si $\sum_{i=1}^d x_i > \theta \rightarrow$ le neurone s'active (sortie = 1), sinon sortie = 0



Inspiration - Neurone biologique - Résumé

3. **Axone** : Une longue fibre nerveuse qui sort du corps cellulaire et transporte le potentiel d'action (signal électrique) du neurone vers d'autres neurones.
 - **Contrepartie artificielle** : Une simple connexion entre neurones (la sortie).
4. **Synapse**: Jonction entre la terminaison de l'axone et la dendrite d'un autre neurone. Les neurotransmetteurs sont libérés à cet endroit pour propager le signal. La force du signal transmis est modulée par la force synaptique.
 - **Contrepartie artificielle** : Les poids d'un neurone artificiel modélisent cette force synaptique. Un poids plus élevé signifie que la sortie du neurone a une plus grande influence sur le neurone suivant.



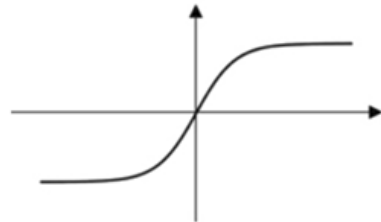
Réseaux neuronaux artificiels

Quelques modifications pour créer un modèle plus puissant :

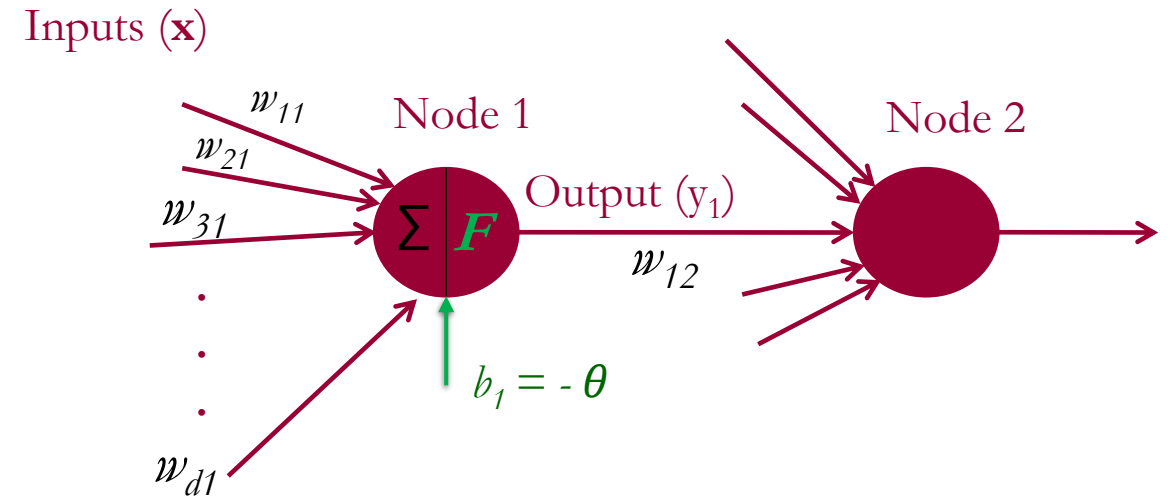
- Pour simplifier, le seuil est déplacé vers les entrées sous la forme d'un paramètre appelé biais (b) :

Si $\sum_{i=1}^d x_i - \theta > 0 \rightarrow$ le neurone s'active (sortie = 1), sinon sortie = 0

- Pour plus de puissance, la fonction d'activation (F) peut être plus complexe qu'une fonction échelon :



- Des poids sont appliqués à toutes les connexions.



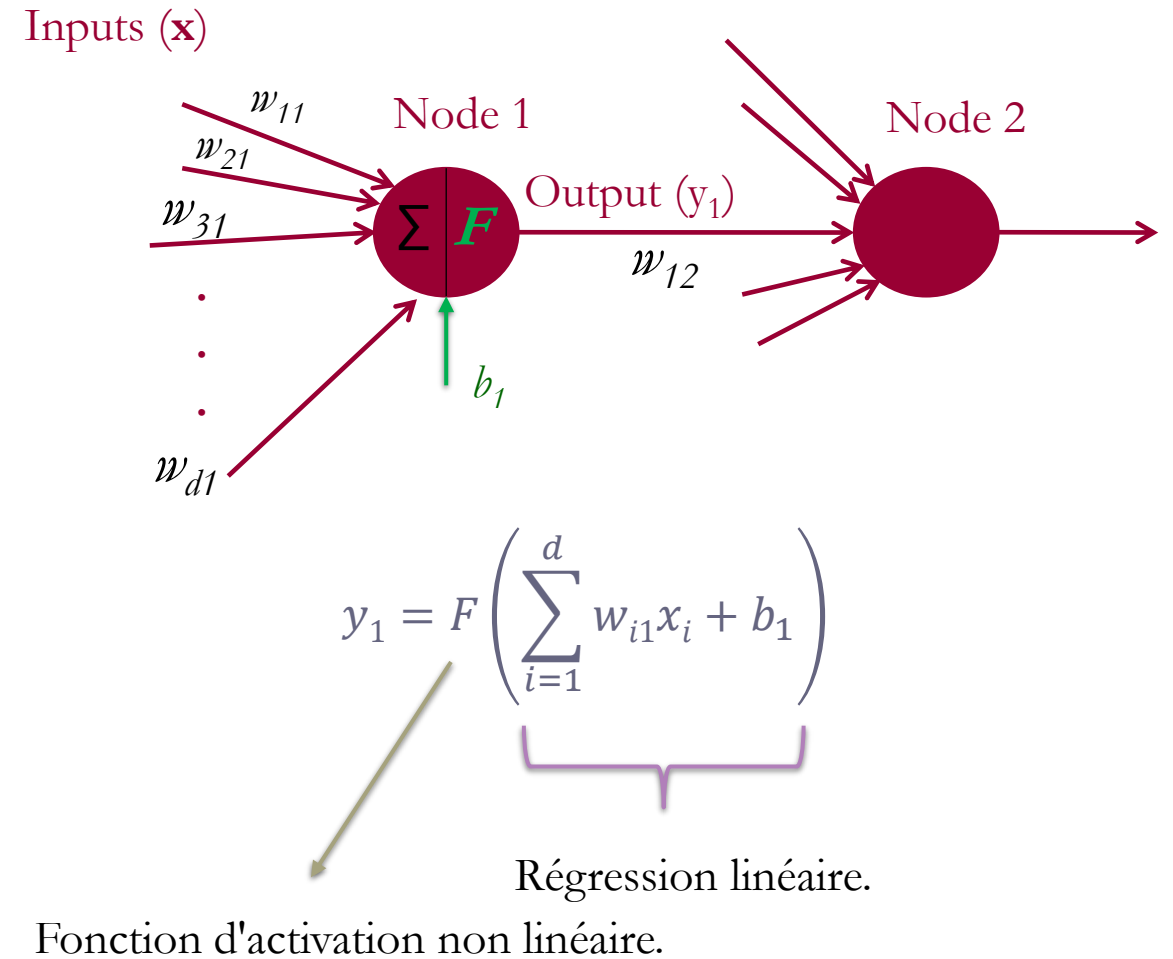
Réseaux neuronaux artificiels

À chaque connexion entrante, un nœud assigne un nombre connu sous le nom de « poids ».

Une entrée est multipliée par les poids, puis sommée.

Les entrées sont multipliées par les poids correspondants, puis sommées.

Ainsi, chaque nœud devient un régresseur non linéaire (effectuant une régression non linéaire).



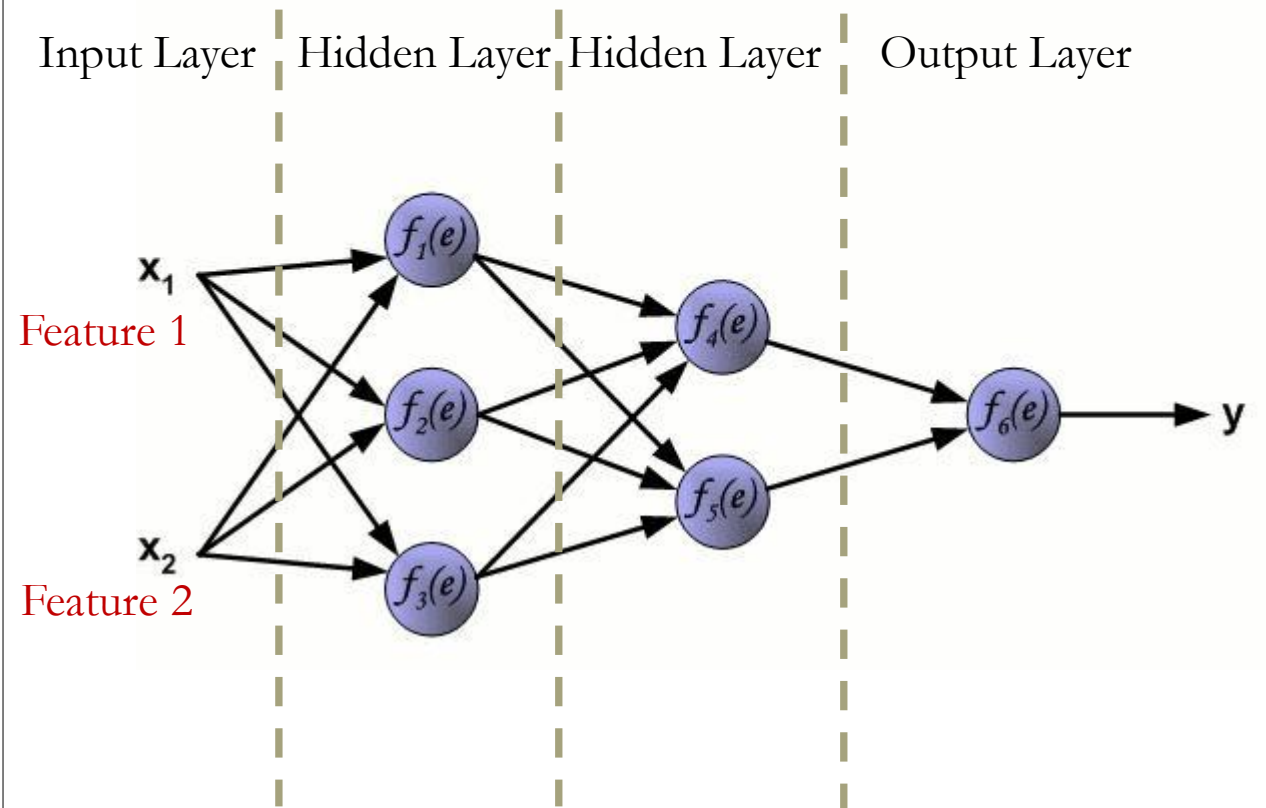
Perceptron multicouche (MLP)

Les réseaux de neurones sont structurés en plusieurs couches, chaque couche générant une nouvelle représentation des données d'entrée.

Chaque nœud effectue une régression non linéaire, et en connectant plusieurs neurones, le réseau devient très efficace pour apprendre des relations complexes.

L'empilement de couches permet de représenter différents niveaux d'abstraction, améliorant la détection de motifs complexes.

Un Perceptron Multicouche (MLP), ou réseau de neurones à propagation avant (Feed-forward neural network) (FFNN), est un réseau où l'information circule dans une seule direction—des nœuds d'entrée, à travers les nœuds cachés, jusqu'aux nœuds de sortie—sans cycles ni boucles.



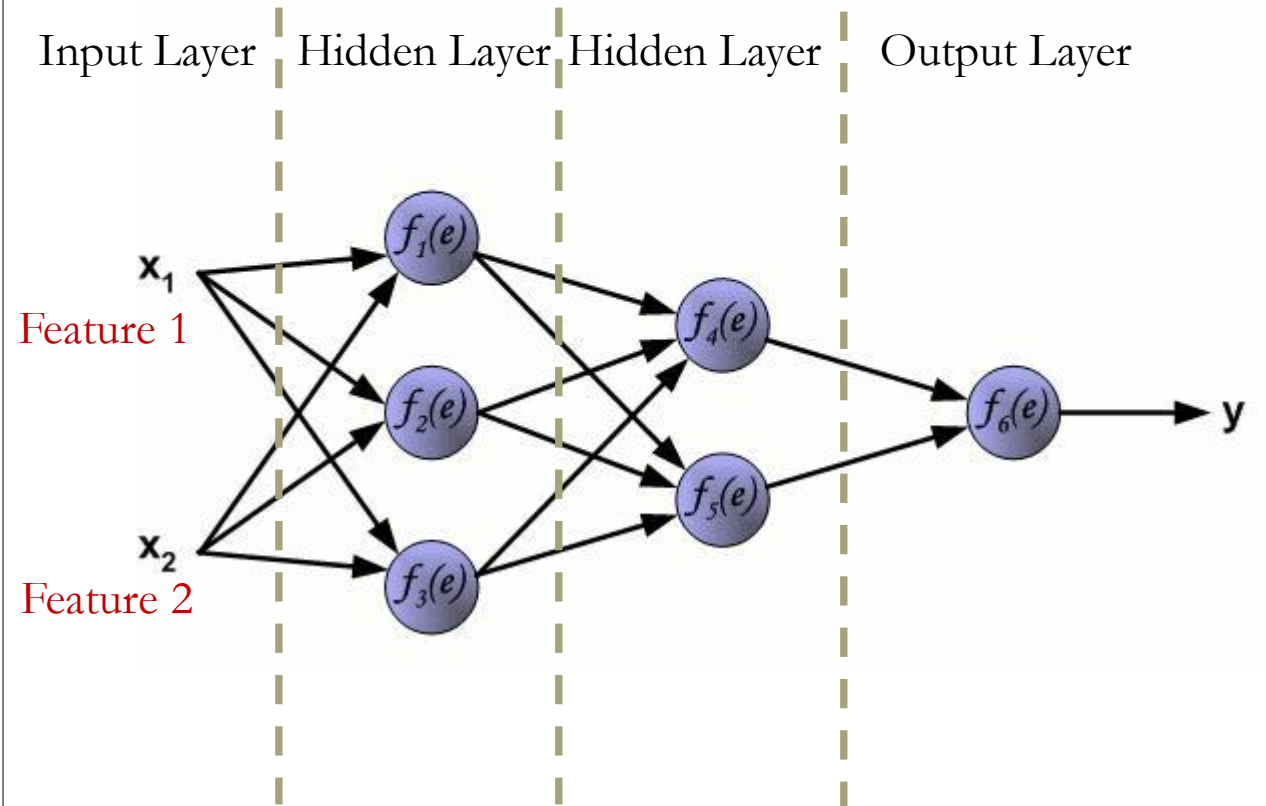
Rétropropagation (Backpropagation)

Considérons un perceptron multicouche (MLP) (FFNN) avec un vecteur de caractéristiques d'entrée 2D $\mathbf{x} = [x_1, x_2]^T$, deux couches cachées, six neurones avec des fonctions d'activation $f_i(e)$ où e est la somme pondérée des entrées du neurone (sorties des couches précédentes), et une sortie y .

Par simplicité, supposons qu'il n'y ait qu'un seul exemple d'entraînement, et nous étendrons plus tard nos calculs à plusieurs exemples d'entraînement.

Nous représentons également les poids reliant le neurone i au neurone j par w_{ij} , et les poids reliant la caractéristique d'entrée x_i au neurone j par $w_{(xi)j}$.

Maintenant, nous voulons trouver une relation entrée/sortie en entraînant (optimisant) les paramètres du réseau de neurones, c'est-à-dire les poids, en utilisant la descente de gradient.



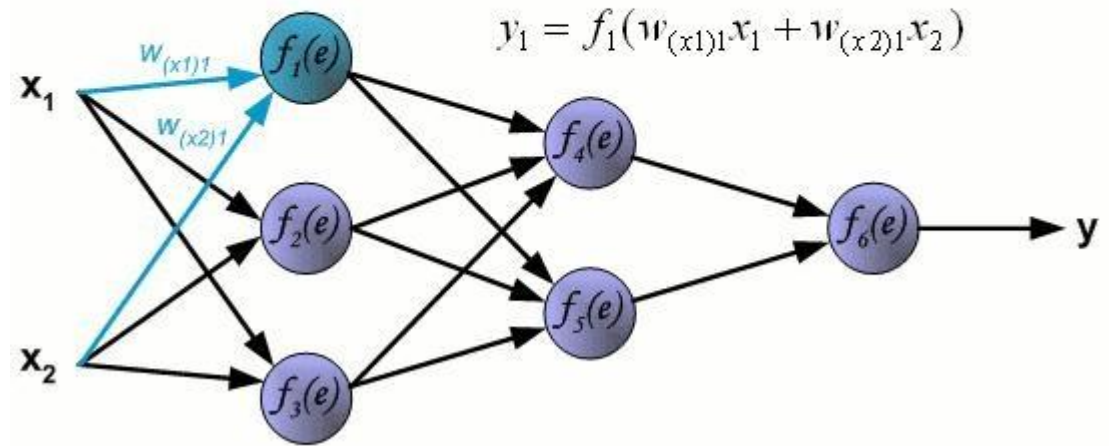
Rétropropagation

Passe avant (Forward Pass)

Tout d'abord, on initialise les poids avec des valeurs aléatoires. Ensuite, nous propageons les entrées vers la sortie, pour calculer la sortie prédite.

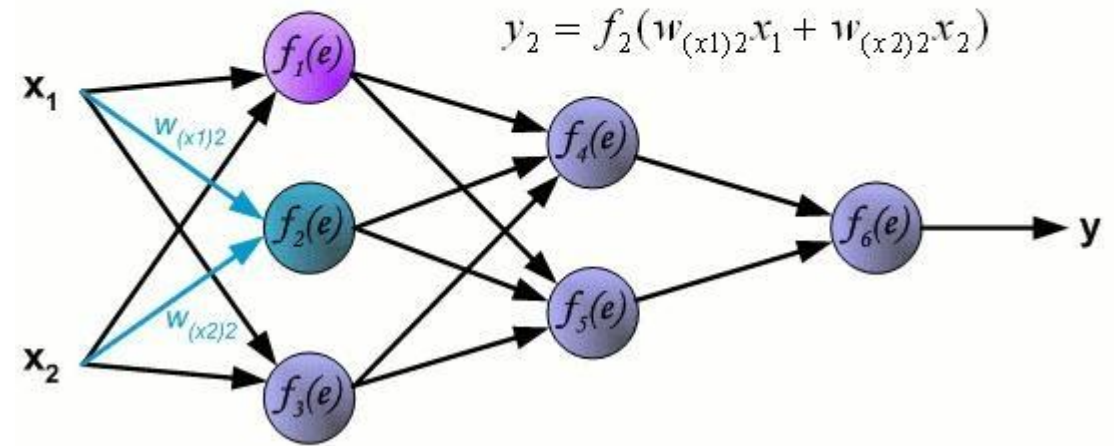
La sortie du neurone i est représentée par y_i .

La sortie du neurone 1 peut être écrite comme suit.



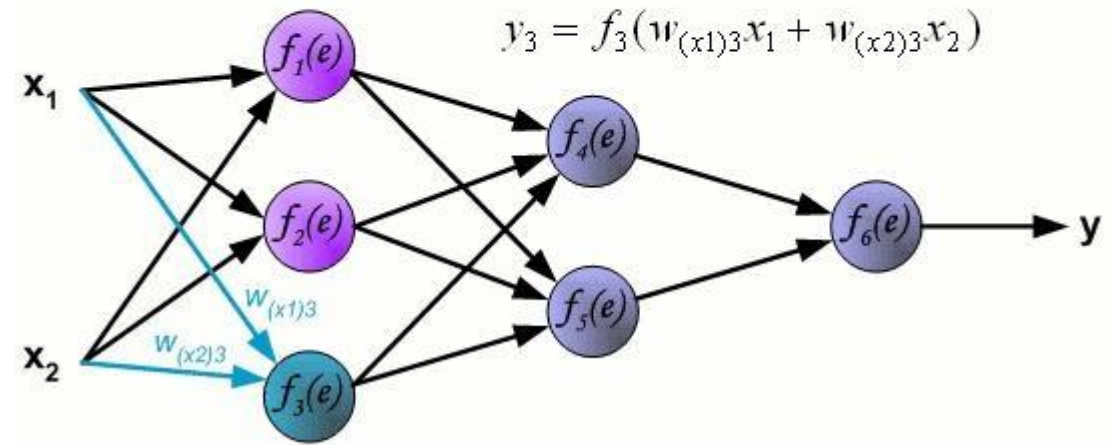
Rétropropagation

Passe avant (Forward Pass)



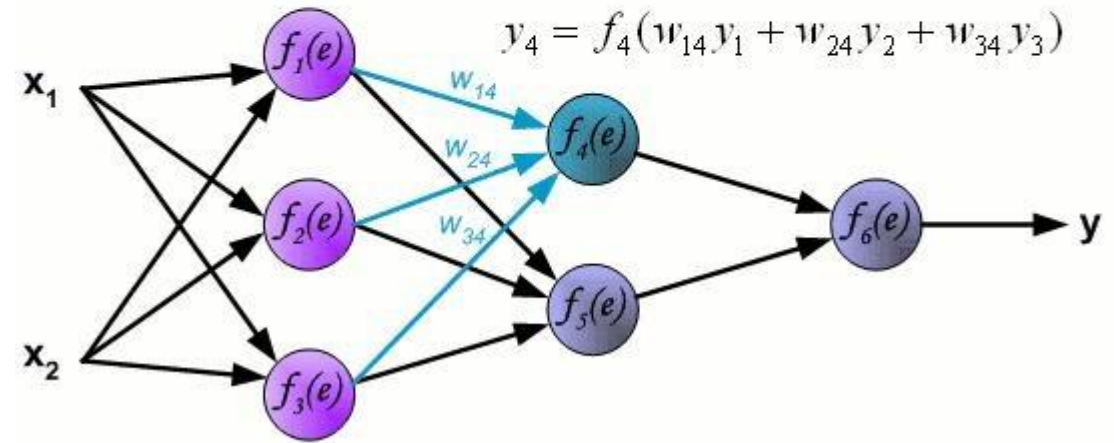
Rétropropagation

Passe avant (Forward Pass)



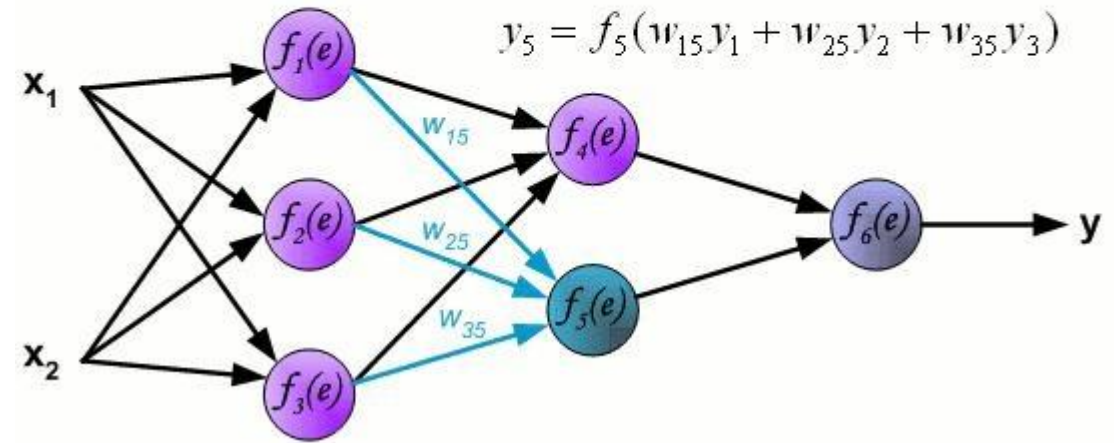
Rétropropagation

Passe avant (Forward Pass)



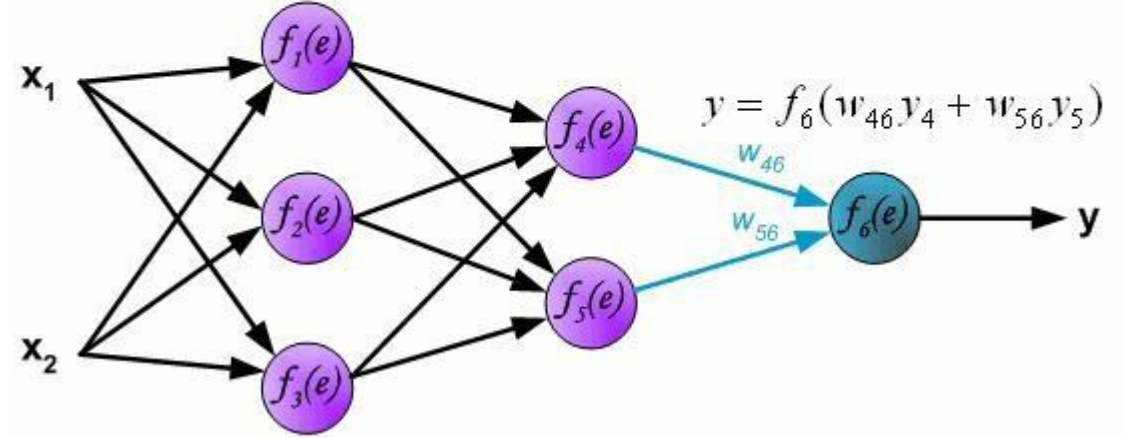
Rétropropagation

Passe avant (Forward Pass)



Rétropropagation

Passe avant (Forward Pass)



Rétropropagation

Considérons la MSE comme fonction de coût :

$$C(\mathbf{w}) = \frac{1}{2}(z - y)^2, \text{ où } z \text{ représente le label désiré.}$$

Les poids de la dernière couche, par exemple w_{46} , peuvent être mis à jour à l'aide de l'algorithme de descente du gradient comme suit :

$$w'_{46} = w_{46} - \eta \nabla C$$

où w'_{46} est le poids mis à jour. Le gradient de C doit être pris par rapport à w_{46} , ce qui peut être obtenu à l'aide des *dérivées partielles* :

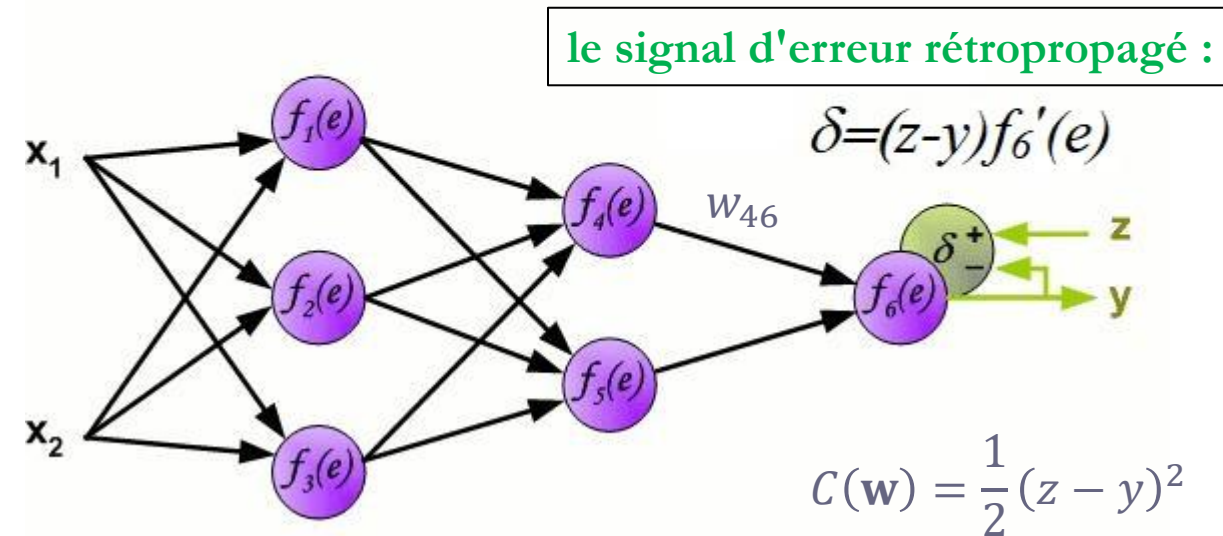
$$\begin{aligned} \frac{dC}{dw_{46}} &= \frac{dC}{dy} \frac{dy}{de_6} \frac{de_6}{dw_{46}} \\ &= -(z - y) \frac{df_6}{de_6} y_4 = -\delta y_4, \text{ où } \delta = -\frac{dC}{dy} \frac{dy}{de_6} = (z - y) \frac{df_6}{de_6} \end{aligned}$$

Rappelons que :

$$e_6 = w_{46}y_4 + w_{56}y_5$$

Ainsi, nous obtenons :

$$w'_{46} = w_{46} + \eta \delta y_4$$



Rétropropagation

Mettons à jour w_{14} :

$$\frac{dC}{dw_{14}} = \left(\frac{dC}{dy} \frac{dy}{de_6} \right) \frac{de_6}{dw_{14}} = -\delta \frac{de_6}{dw_{14}}$$

Rappelez-vous, ici :

$$e_6 = w_{46}y_4 + w_{56}y_5$$

Ainsi,

$$\frac{dC}{dw_{14}} = -\delta \left(w_{46} \frac{dy_4}{dw_{14}} + w_{56} \frac{dy_5}{dw_{14}} \right) = -\delta \left(w_{46} \frac{df_4}{de_4} \frac{de_4}{dw_{14}} \right)$$

Rappelez-vous, ici :

$$e_4 = w_{14}y_1 + w_{24}y_2 + w_{34}y_3$$

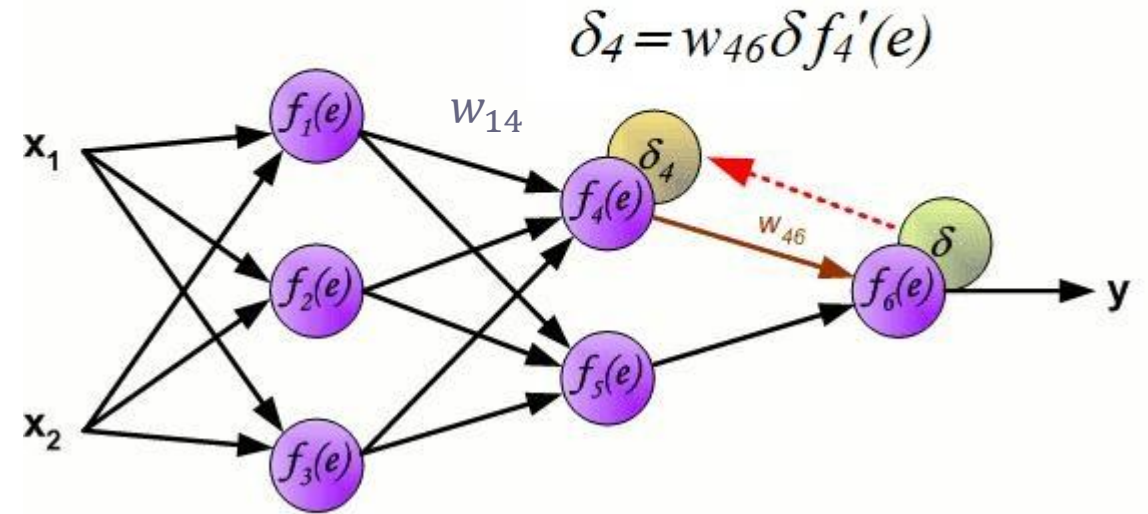
Ainsi,

$$\frac{dC}{dw_{14}} = - \left(\delta w_{46} \frac{df_4}{de_4} \right) y_1 = -\delta_4 y_1$$

Nous avons donc :

$$w'_{14} = w_{14} + \eta \delta_4 y_1$$

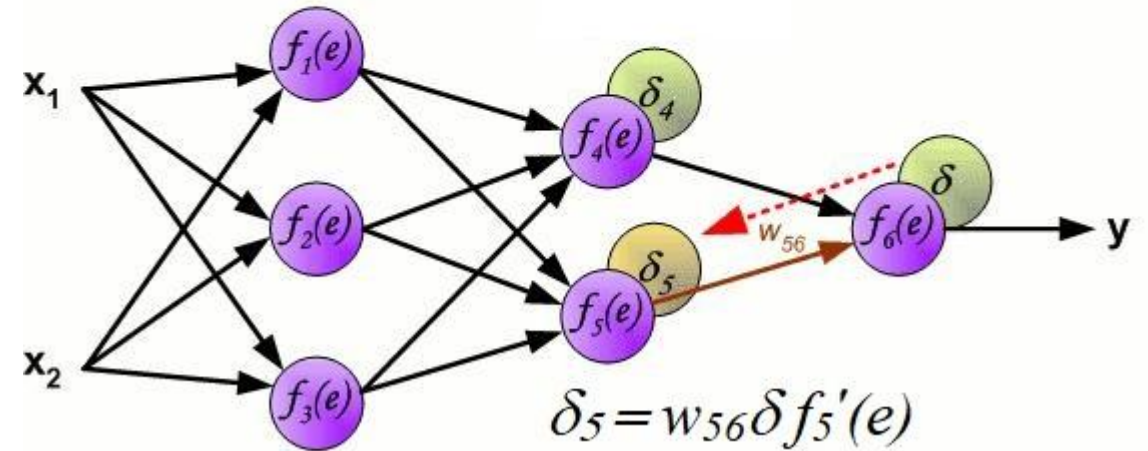
Important à noter : δ_4 : signal d'erreur à rétropropager.



Rétropropagation

Passe arrière (Backward Pass)

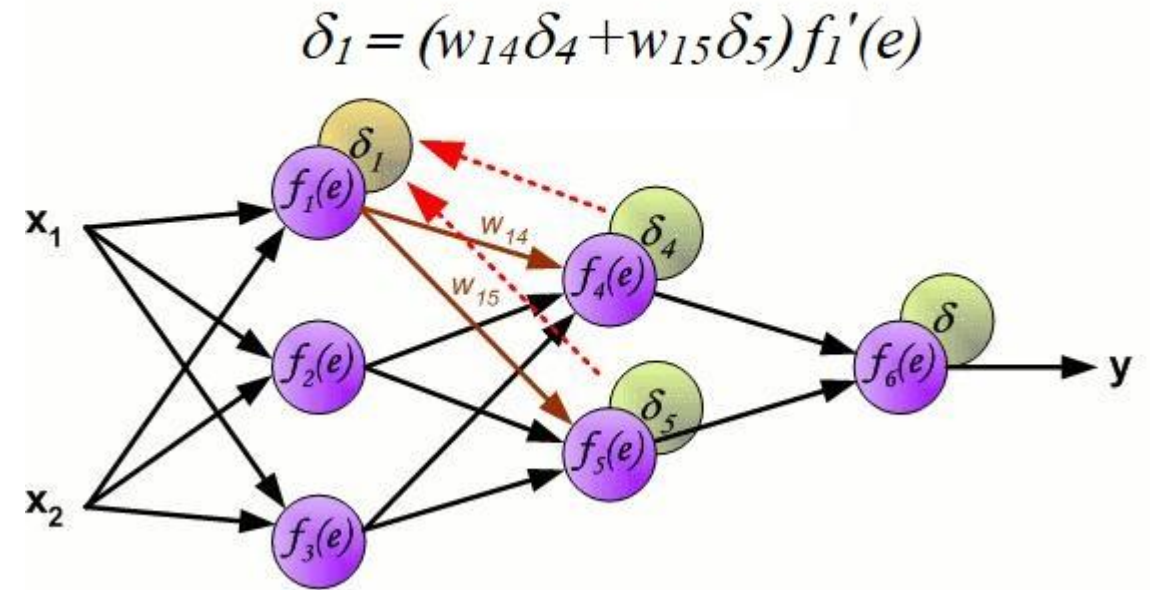
Nous pouvons donc voir qu'en définissant le terme d'erreur delta, la mise à jour des poids devient facile. Au lieu de calculer les dérivées à chaque fois pour chaque poids, nous devons rétropropager ce terme d'erreur. Comment ? Nous le multiplions par le poids correspondant et le gradient local de la fonction d'activation.



Rétropropagation

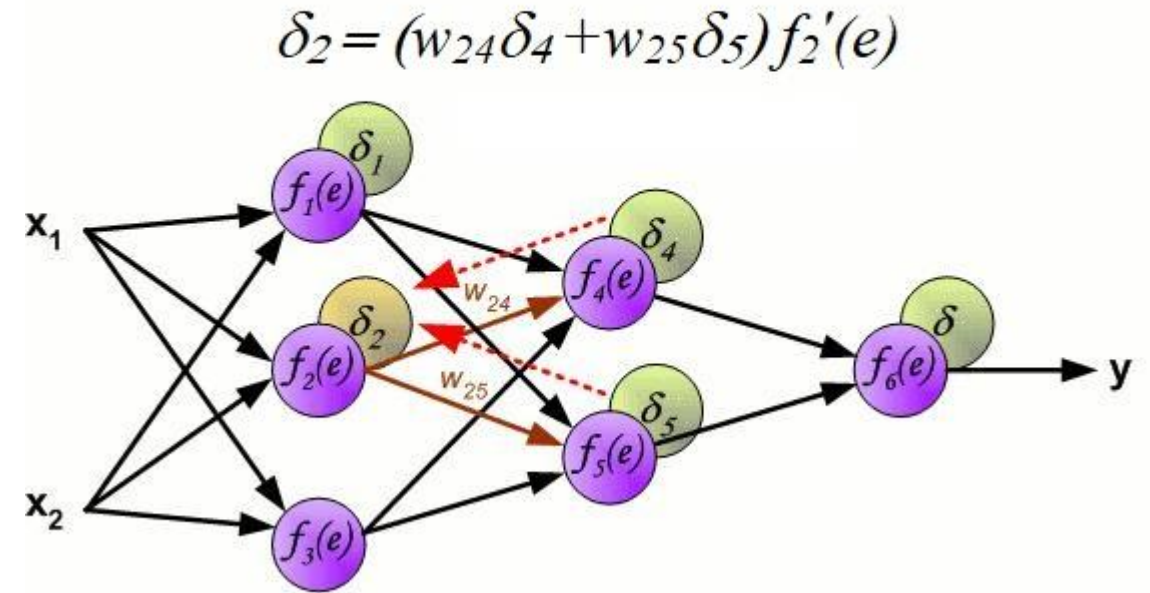
Passe arrière (Backward Pass)

Nous avons ici deux termes d'erreur à rétropropager.



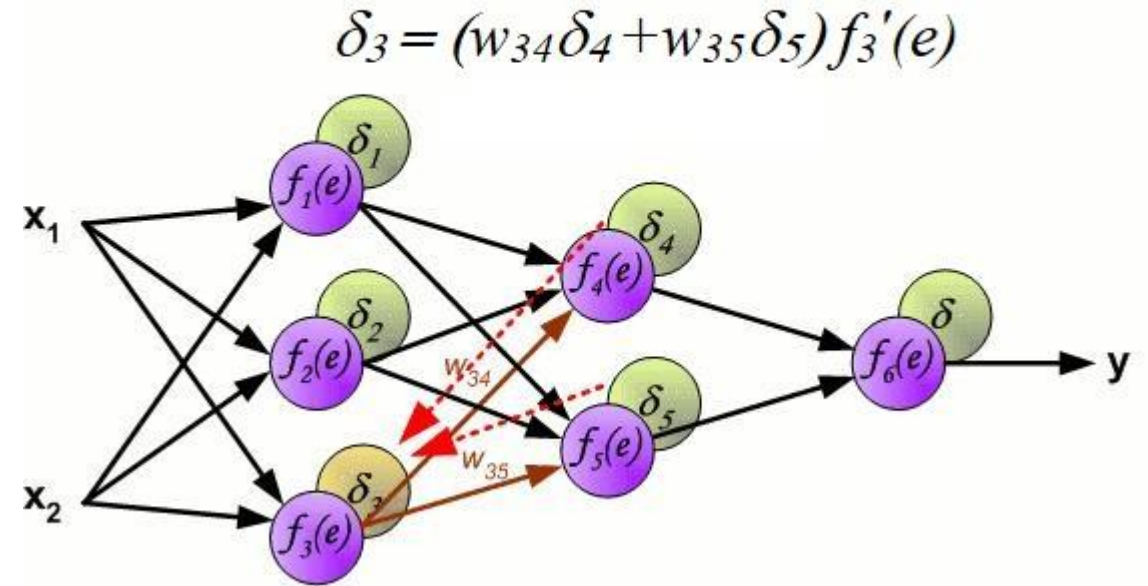
Rétropropagation

Passe arrière (Backward Pass)



Rétropropagation

Passe arrière (Backward Pass)

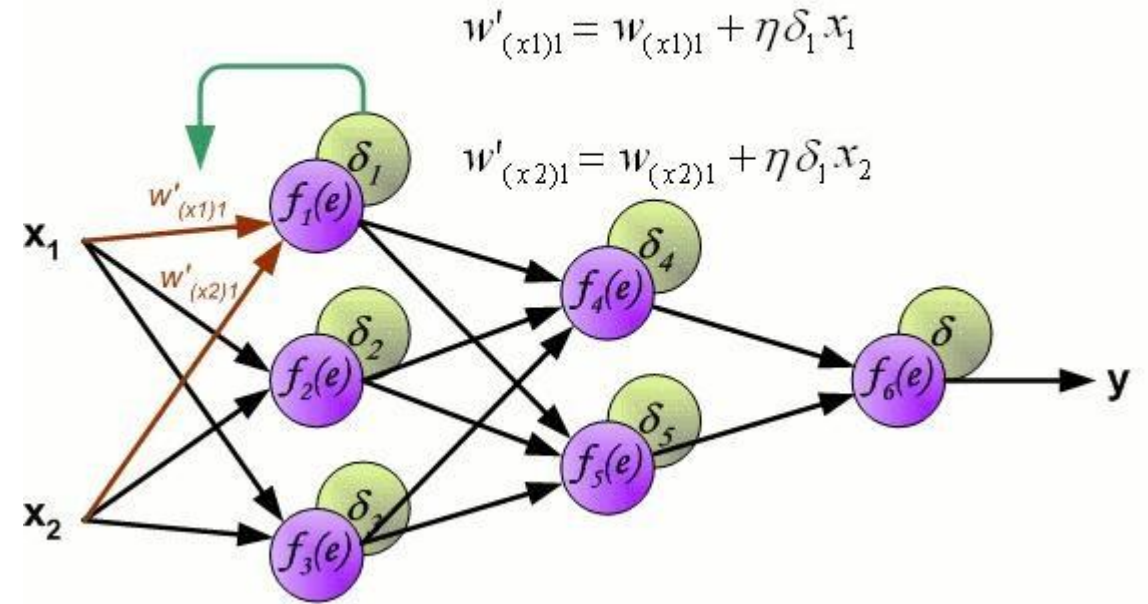


Rétropropagation

Mise à jour des poids :

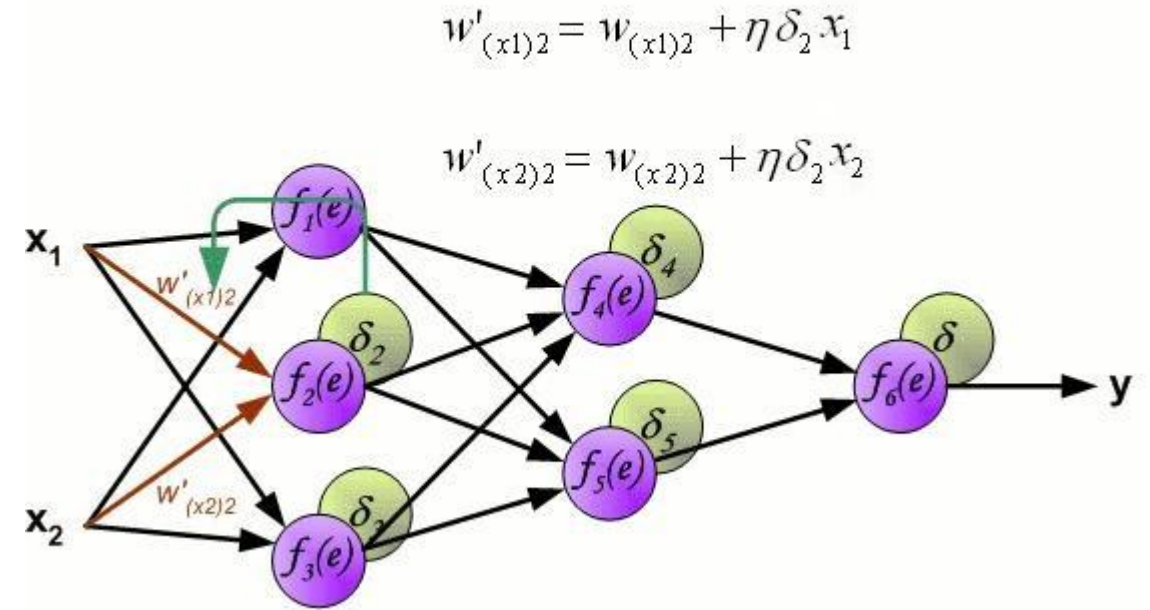
Nous pourrions mettre à jour les poids dès que nous avons calculé le signal d'erreur correspondant. Mais, généralement, nous le faisons systématiquement après avoir calculé tous les termes d'erreur.

$$w'_{(x_1)1} = w_{(x_1)1} + \eta \delta_1 x_1$$



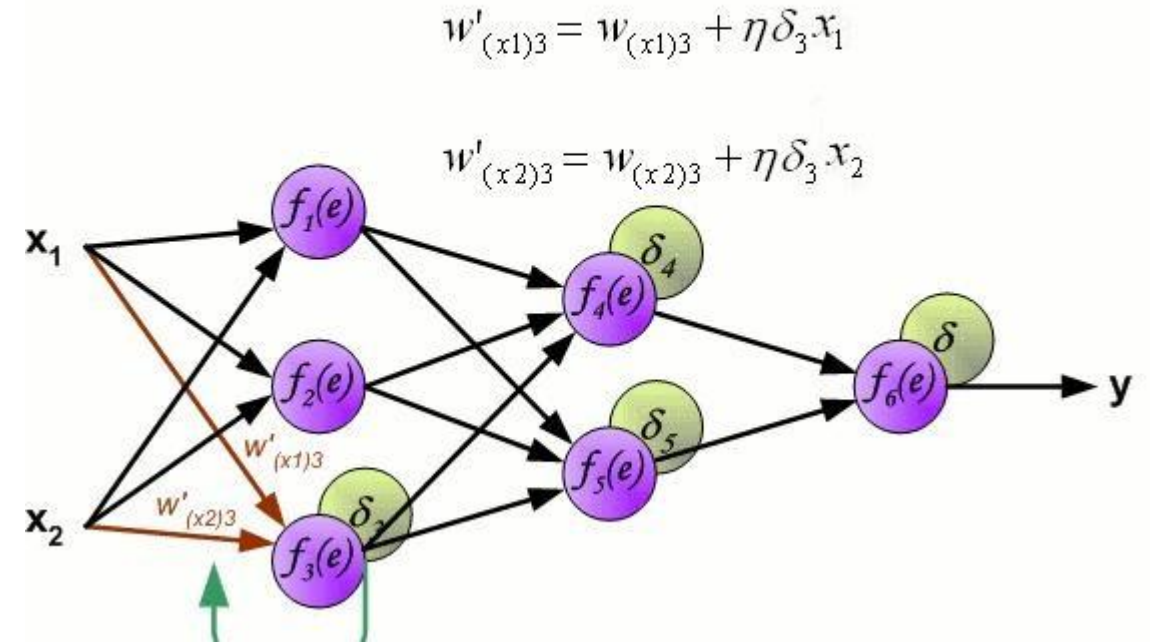
Rétropropagation

Mise à jour des poids



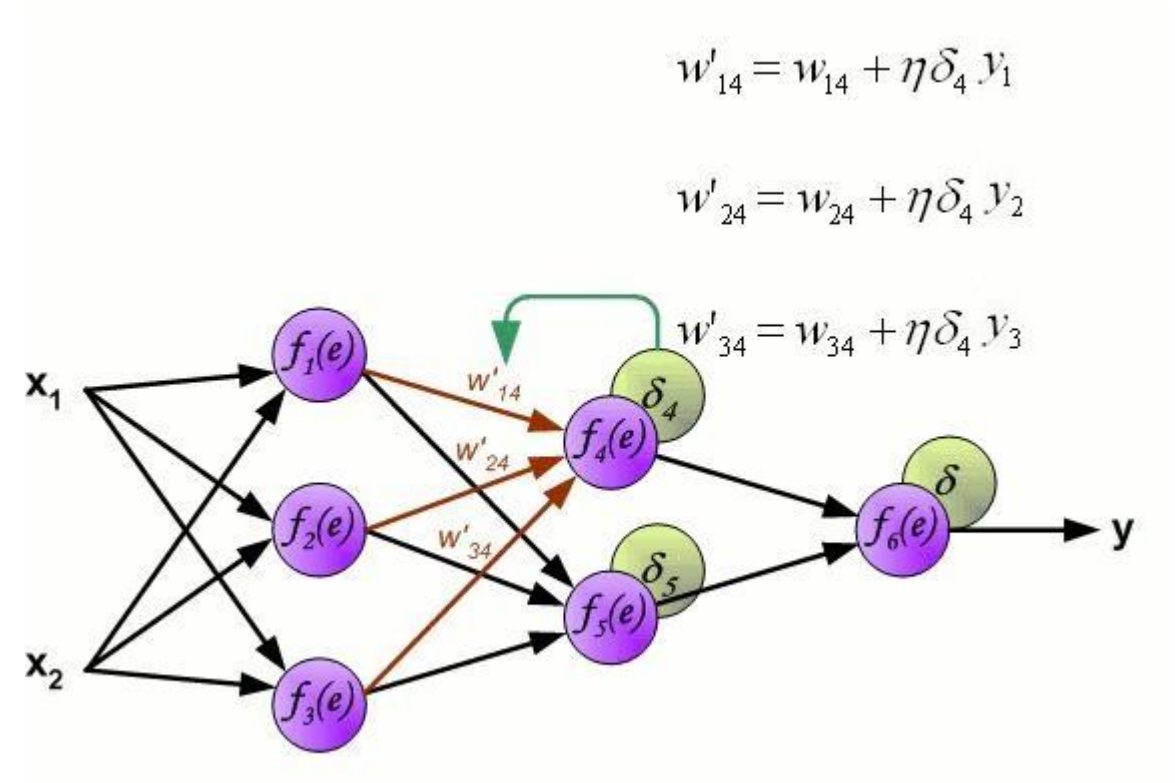
Rétropropagation

Mise à jour des poids



Rétropropagation

Mise à jour des poids

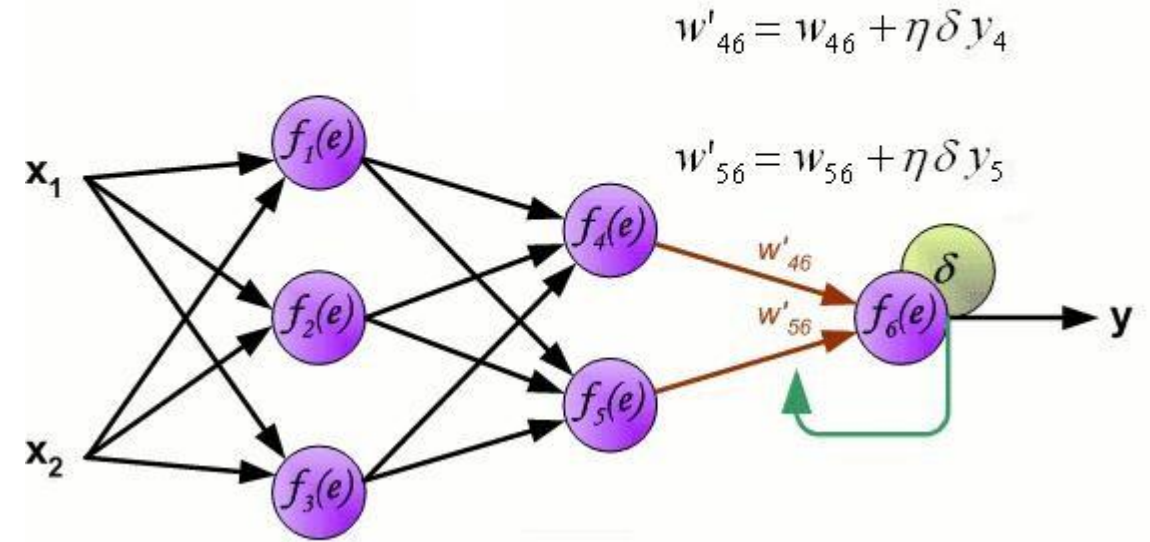


Mise à jour des poids



Rétropropagation

Mise à jour des poids



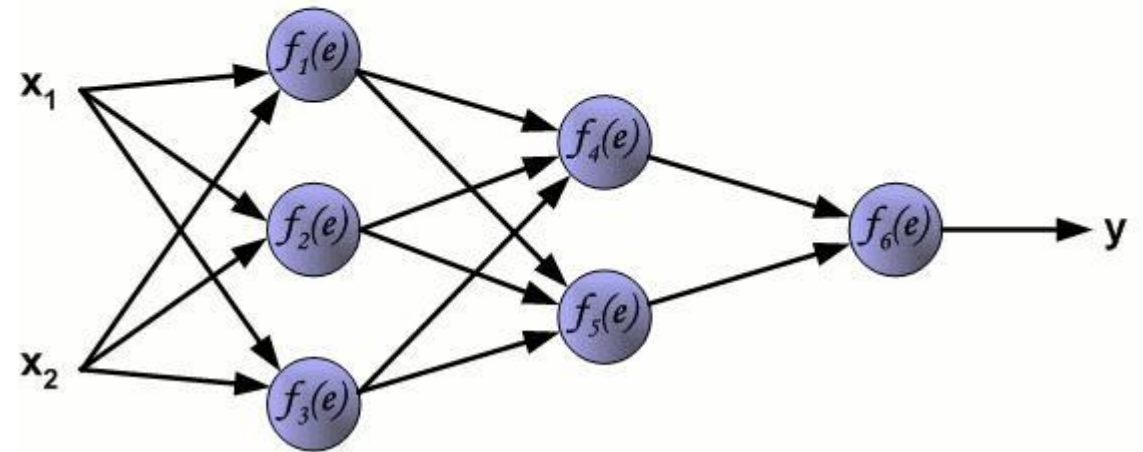
Rétropropagation

Résumé :

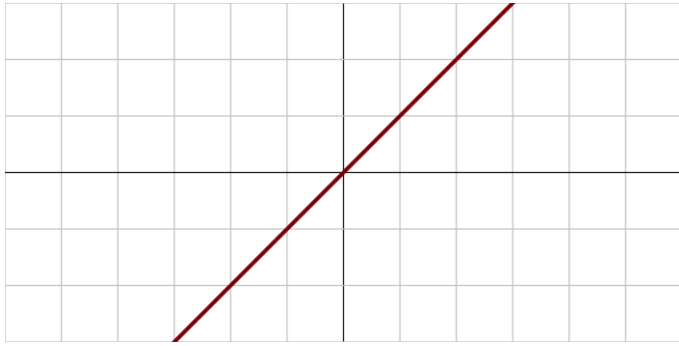
Passe avant (Forward Pass) : calculer toutes les sorties neuronales et l'erreur totale.

Passe arrière (Backward Pass) : calculer le signal d'erreur (le terme δ) pour la couche de sortie. Ensuite, propager ce signal d'erreur vers l'arrière à travers le réseau, de la dernière couche à la première (multiplier par les poids correspondants et la dérivée de la fonction d'activation locale). Stocker le signal d'erreur correspondant.

Mise à jour des poids (Weight Update) : mettre à jour tous les poids à l'aide des sorties calculées lors du passage avant et du signal d'erreur calculé et stocké lors du passage arrière.



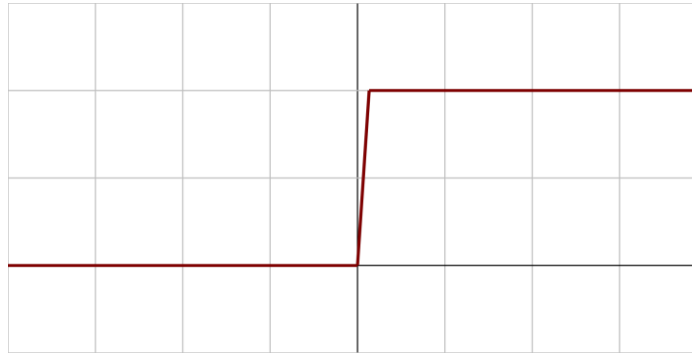
Linéaire



Adapté à la régression (estimation d'une fonction continue)
(dans la dernière couche d'un réseau de neurones)

Step (échelon)

$$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

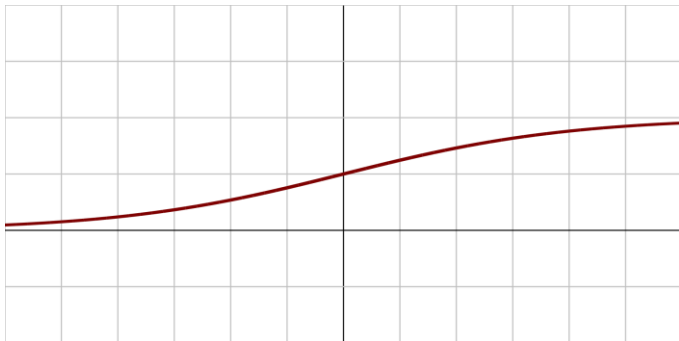


Adapté à la classification
(dans la dernière couche)

Fonction d'activation (F)

En fonction de la manière dont nous souhaitons que le nœud du réseau de neurones artificiel (ANN) traite les données d'entrée, de l'étendue de la sortie souhaitée, de la profondeur du réseau, etc., il existe différents choix de fonctions d'activation.

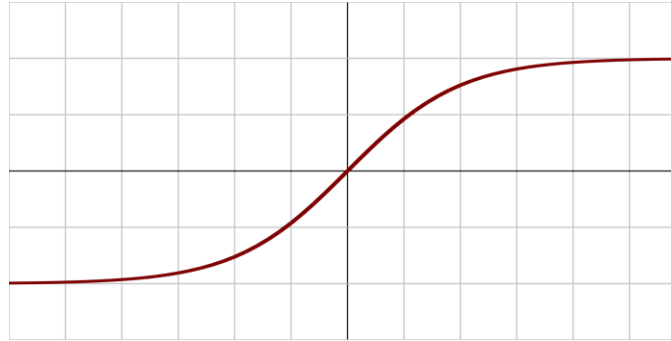
Sigmoïde $\sigma(x) \doteq \frac{1}{1 + e^{-x}}$



Dérivable. Adapté à la plupart des problèmes dans les couches cachées.

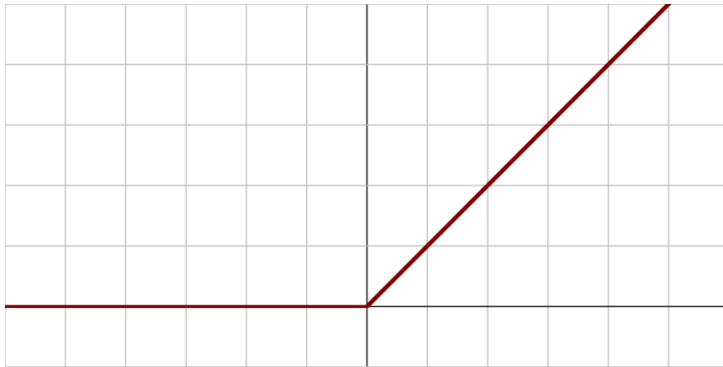
Tangente hyperbolique (tanh)

$$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

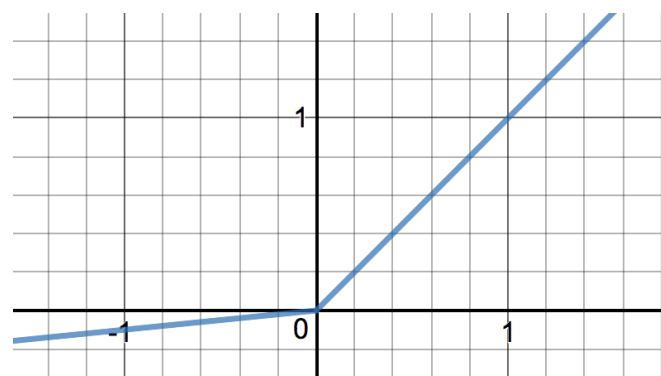


Dérivable. Adapté à la plupart des problèmes dans les couches cachées.

Rectified linear unit (Relu)



Leaky Relu



Bon pour les réseaux de neurones profonds (avec de nombreuses couches cachées).

$$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} \\ = \max(0, x) = x \mathbf{1}_{x>0}$$

$$\begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

Le ReLU et sa version Leaky aident à surmonter le problème du vanishing gradient. Lors de l'utilisation de fonctions comme la sigmoïde, le gradient calculé lors de la rétropropagation à travers plusieurs dérivées partielles peut devenir très petit dans les réseaux profonds, ralentissant l'apprentissage dans les premières couches.

Fonction d'activation (F)

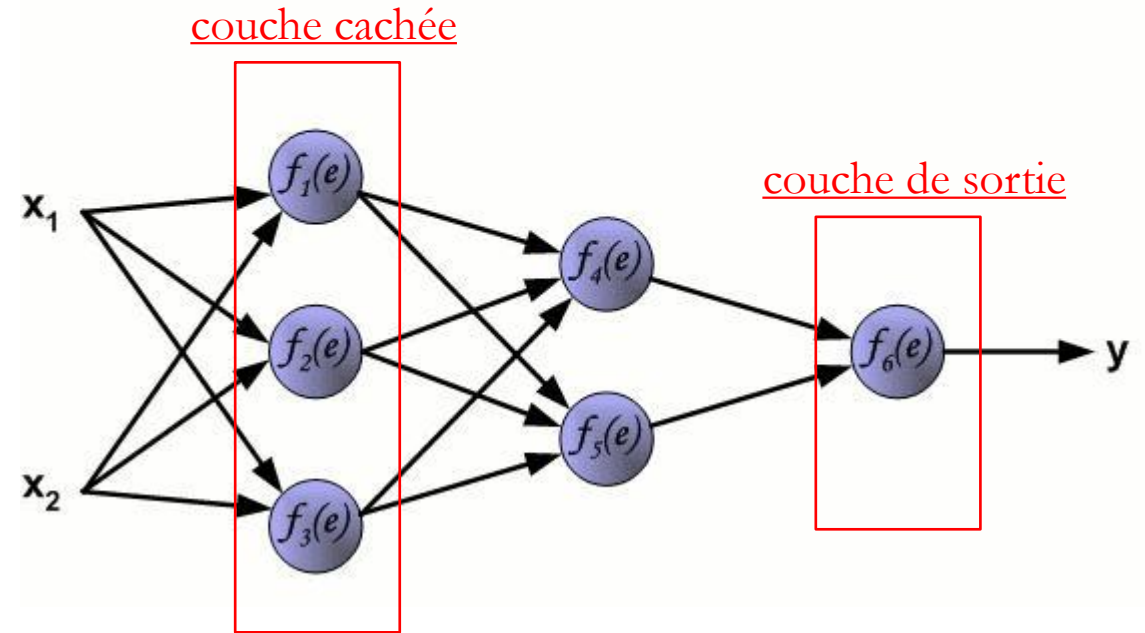
En fonction de la manière dont nous souhaitons que le nœud du réseau de neurones artificiel (ANN) traite les données d'entrée, de l'étendue de la sortie souhaitée, de la profondeur du réseau, etc., il existe différents choix de fonctions d'activation.

Théorème d'approximation universelle

Le **théorème d'approximation universelle** (Hornik et al. 1989, Cybenko, 1989) stipule qu'un réseau neuronal feedforward (à propagation avant) avec une couche de sortie linéaire et au moins une couche cachée avec une fonction d'activation non linéaire (sigmoïde ou tanh par exemple) peut approximer n'importe quelle fonction continue sur un ensemble fermé et limité dans $\mathbb{R}^d$, avec n'importe quelle quantité d'erreur non nulle souhaitée, à condition que le réseau ait suffisamment de nœuds cachés.

Cette théorie a été étendue à la fonction d'activation ReLU (Leshno et al., 1993).

Donc, en théorie, nous pouvons approximer n'importe quelle fonction à l'aide de réseaux neuronaux.

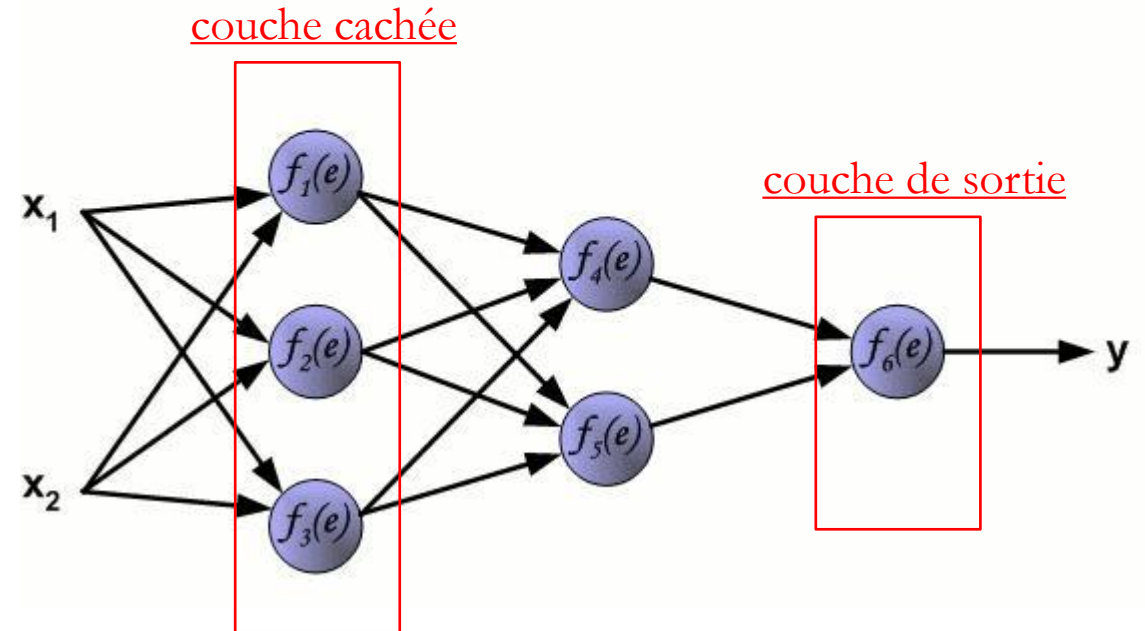


Théorème d'approximation universelle

Même si le théorème d'approximation universelle semble prometteur, un réseau à propagation avant peut ne pas apprendre une fonction, même s'il peut théoriquement la représenter. **Pourquoi ?**

Nous savons qu'il existe une solution, mais nous ne savons pas laquelle. Pour trouver la solution, nous utilisons une approche d'optimisation. Cependant, les procédures d'optimisation utilisées pour entraîner les réseaux de neurones ne garantissent pas de trouver les paramètres corrects correspondant à une fonction donnée.

Donc, nous avons besoin de méthodes d'optimisation efficaces, de bonnes stratégies d'initialisation des paramètres et de données de qualité pour améliorer l'apprentissage du réseau de neurones.



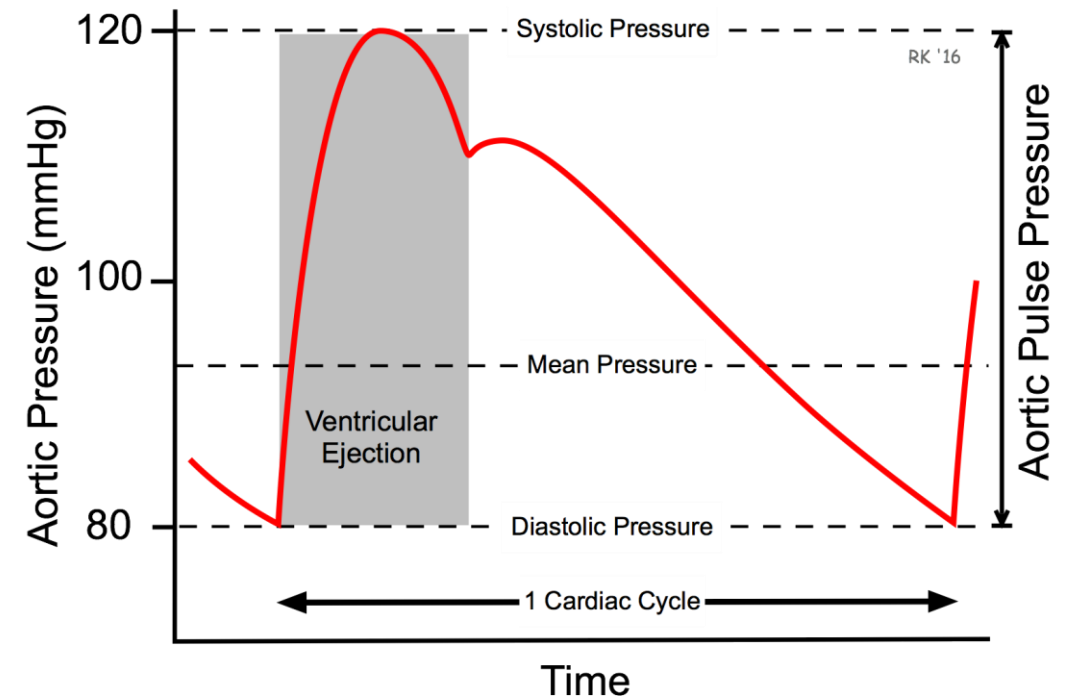
Estimation de la pression artérielle à partir du PPG

La pression artérielle (blood pressure) (BP) est définie comme la force exercée par le sang sur les parois des artères.

Elle est quantifiée par les pressions minimales et maximales au cours de chaque cycle cardiaque, appelées tension systolique et tension diastolique.

Les valeurs typiques de la tension artérielle sont 120 mmHg et 80 mmHg, pour les pressions systolique et diastolique.

Les millimètres de mercure (mmHg) sont l'unité conventionnelle utilisée pour exprimer la tension artérielle.

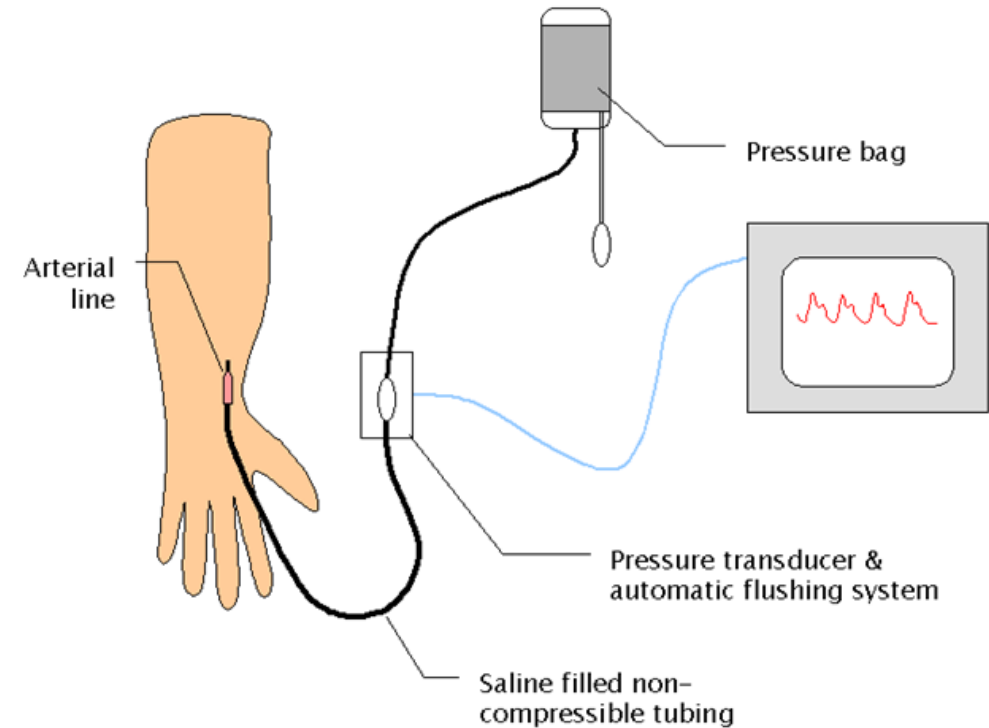


Un pouls de pression artérielle aortique avec des pressions systolique et diastolique illustrées comme les valeurs de pression maximale et minimale au sein du cycle cardiaque.

<https://www.cvphysiology.com/Blood%20Pressure/BP002>

Estimation de la pression artérielle à partir du PPG

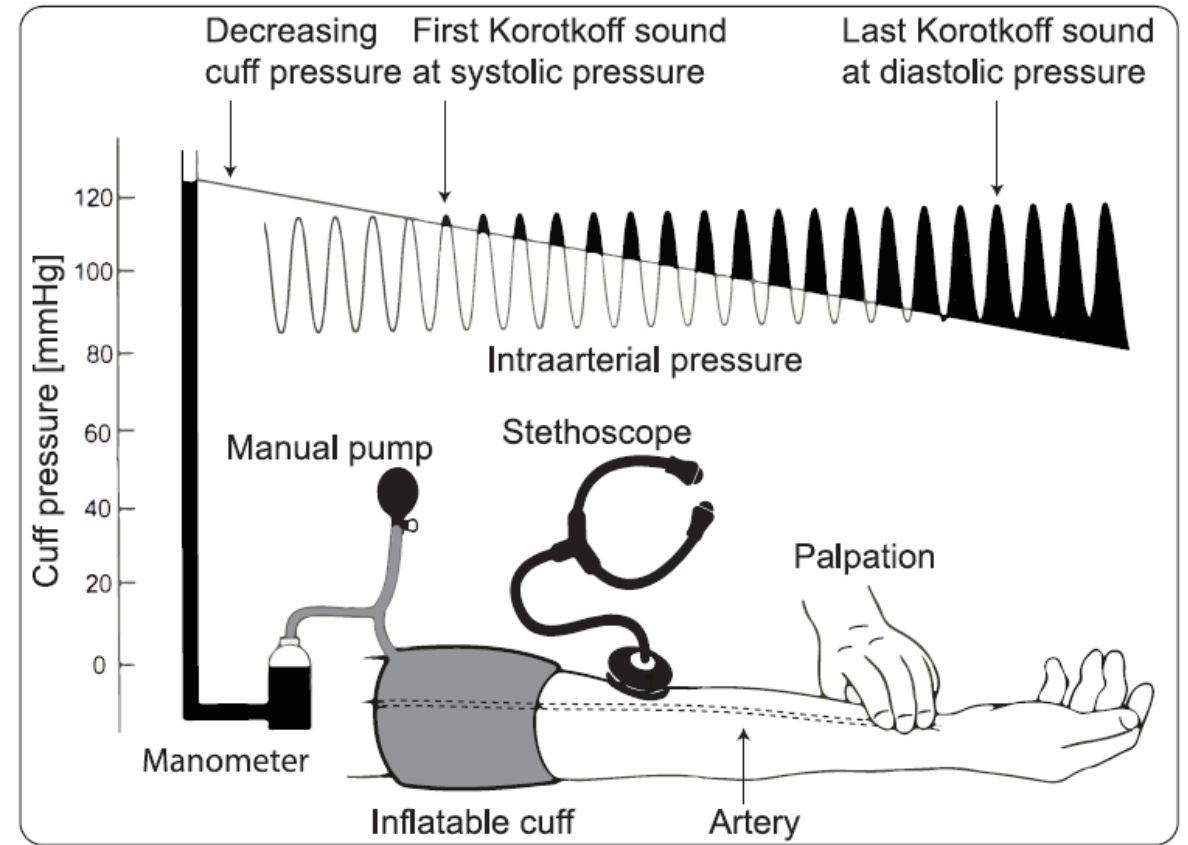
Les méthodes de référence pour la mesure de la tension artérielle sont le cathéter invasif placé dans l'artère (risqué et difficile) et la méthode auscultatoire (qui requiert une certaine expertise).



Mesure invasive de la tension artérielle. (BP).
https://bmet.fandom.com/wiki/Invasive_Blood_Pressure

Estimation de la pression artérielle à partir du PPG

Dans la méthode auscultatoire, un brassard est placé autour du bras à la même hauteur que le cœur et est gonflé jusqu'à ce que l'artère soit complètement obstruée. Le brassard est ensuite lentement dégonflé par un examinateur qui écoute les bruits de Korotkoff à l'aide d'un stéthoscope afin d'identifier la pression artérielle systolique (PAS) et la pression artérielle diastolique (PAD).



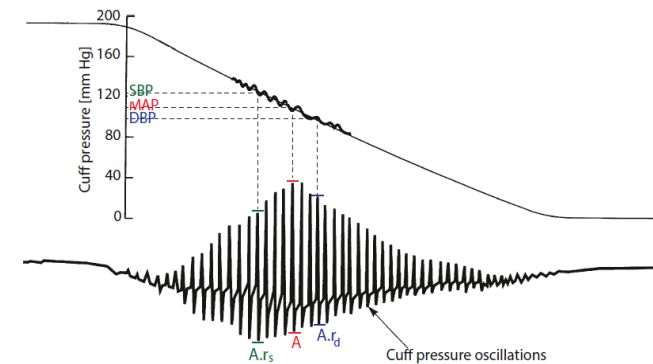
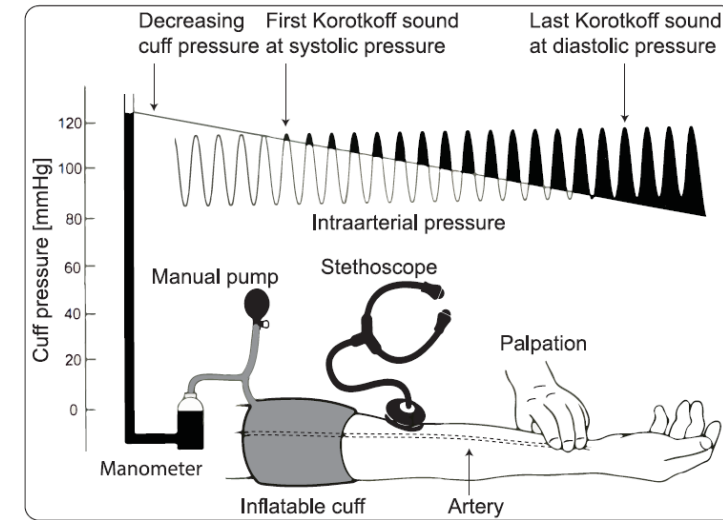
Forouzanfar, Mohamad, et al. "Oscillometric blood pressure estimation: past, present, and future." IEEE reviews in biomedical engineering 8 (2015): 44-63.

Estimation de la pression artérielle à partir du PPG

Parmi les techniques non invasives, les appareils oscillométriques à brassard sont les plus précis.

L'approche oscillométrique utilise un capteur de pression au lieu d'un stéthoscope pour mesurer les variations de pression dans le brassard et certains algorithmes d'analyse pour estimer les pressions systolique et diastolique.

Cependant, l'utilisation du brassard implique des inconvénients.

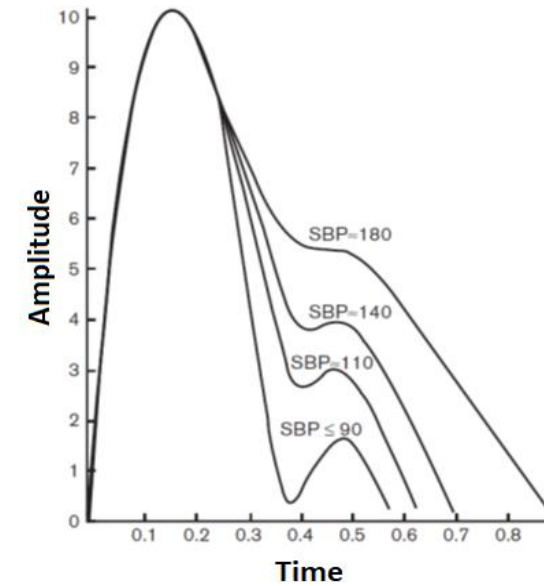


Forouzanfar, Mohamad, et al. "Oscillometric blood pressure estimation: past, present, and future." IEEE reviews in biomedical engineering 8 (2015): 44-63.

Estimation de la pression artérielle à partir du PPG

Il a été démontré que la morphologie du pouls PPG mesuré au doigt ou au poignet varie en fonction des valeurs de la tension artérielle.

Nous voulons concevoir un MLP pour l'estimation des tensions systolique et diastolique à partir de la morphologie du pouls PPG.

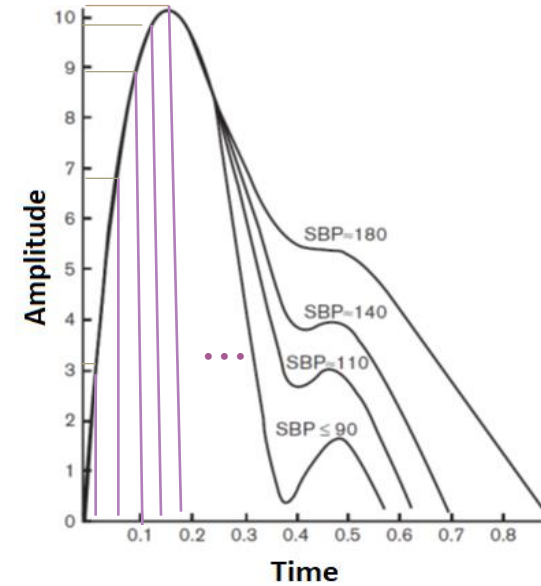


La morphologie du pouls PPG varie en fonction de la pression artérielle.

Estimation de la pression artérielle à partir du PPG

Comment former un vecteur de données d'entrée, (vecteur de caractéristiques) ?

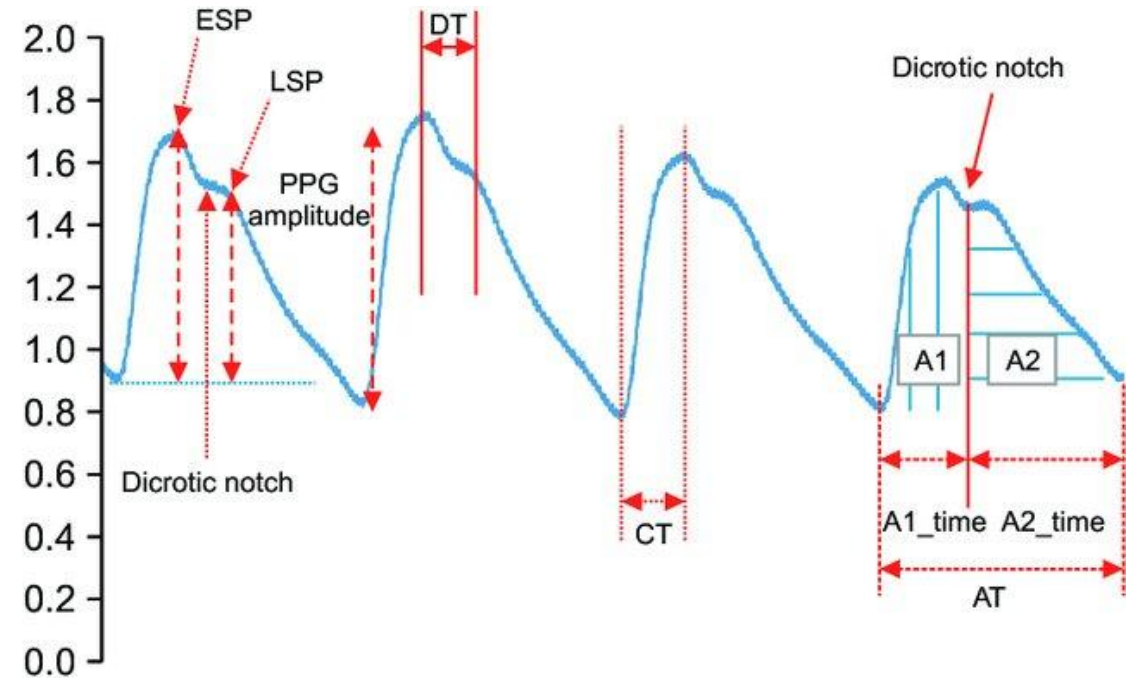
Nous pouvons **échantillonner uniformément** les pouls PPG **bruts** et fournir les valeurs d'amplitude à un MLP pour apprendre la relation entre l'entrée (la morphologie du pouls) et la sortie (la pression artérielle systolique/diastolique).



Échantillonnage uniforme de pouls PPG.

Estimation de la pression artérielle à partir du PPG

Une autre approche consiste à extraire les caractéristiques de chaque cycle cardiaque PPG (pouls).



Quelques caractéristiques possibles de la morphologie de l'impulsion PPG. Pour plus de clarté, les différentes caractéristiques sont représentées sur des impulsions PPG différentes.

Ahn, J. M. (2017). New aging index using signal features of both photoplethysmograms and acceleration plethysmograms. Healthcare informatics research, 23(1), 53-59.

Estimation de la pression artérielle à partir du PPG

Vecteur d'entrée (vecteur de caractéristiques) :

Chaque exemple d'entraînement formera un vecteur de caractéristiques extraites d'un cycle cardiaque PPG (pouls).

Sortie :

Les valeurs de la pression artérielle systolique et diastolique.

Labels :

Les étiquettes peuvent être des estimations de la pression artérielle obtenues par une technique invasive.

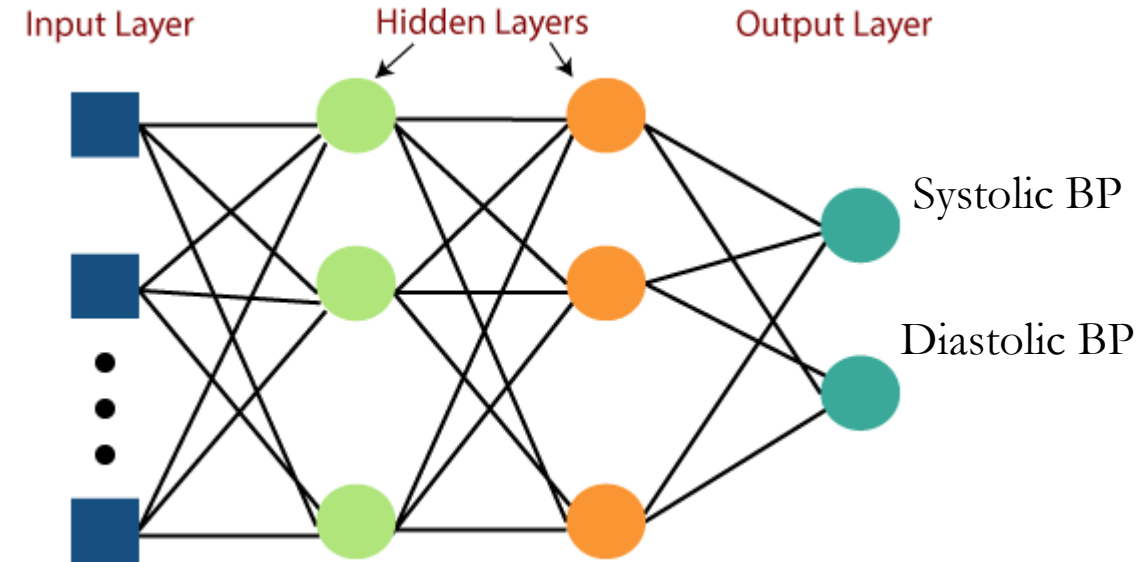
Fonction de coût :

Nous pouvons choisir l'erreur quadratique moyenne (MSE) comme fonction de coût. Étant donné que nous avons deux sorties, nous pouvons utiliser le coût total moyen.

Hyperparamètres :

Un hyperparamètre est un paramètre choisi par l'humain pour contrôler le processus d'apprentissage, et non un paramètre que le modèle apprend lui-même.

Nous devons également déterminer le nombre de couches cachées, le nombre de neurones dans chaque couche cachée, les fonctions d'activation, ainsi que l'algorithme d'optimisation et ses paramètres.



MLP

Sélection des hyperparamètres

(Règles empiriques)

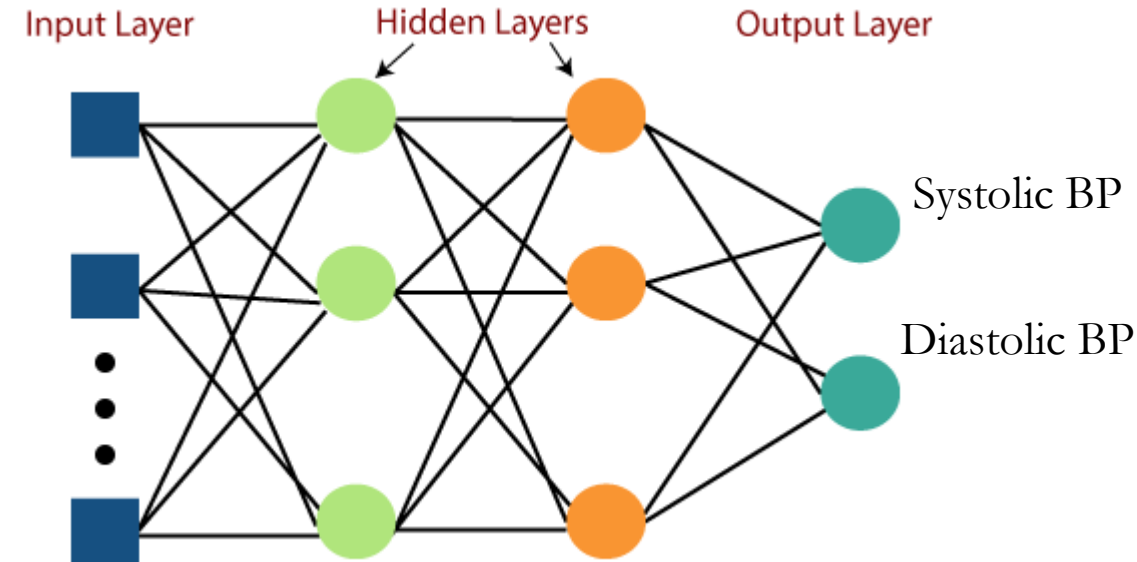
Ici, nous pouvons choisir une fonction d'activation linéaire pour les neurones de la couche de sortie (parce que les sorties sont continues et ne sont pas limitées entre 0 et 1 ou -1 et 1).

Pour le reste des neurones, nous avons choisi la fonction d'activation tanh qui est différentiable, dont la plage est limitée (pour éviter des sorties trop importantes) et qui est comprise entre -1 et 1 (pour fournir des valeurs de sortie à la fois positives et négatives et donc renforcer les capacités du réseau).

Pour l'algorithme d'optimisation, nous avons choisi Adam, qui s'est avéré performant dans de nombreuses tâches de ML.

Nous avons choisi deux couches pour disposer d'une puissance de modélisation suffisante tout en conservant un modèle simple (rappelez-vous la théorie de l'approximation : une seule couche cachée peut suffire).

En supposant que notre dimension d'entrée est de 100, la première couche cachée aura 100 neurones, et pour la deuxième couche nous utilisons une règle empirique : $\sqrt{100} = 10$.

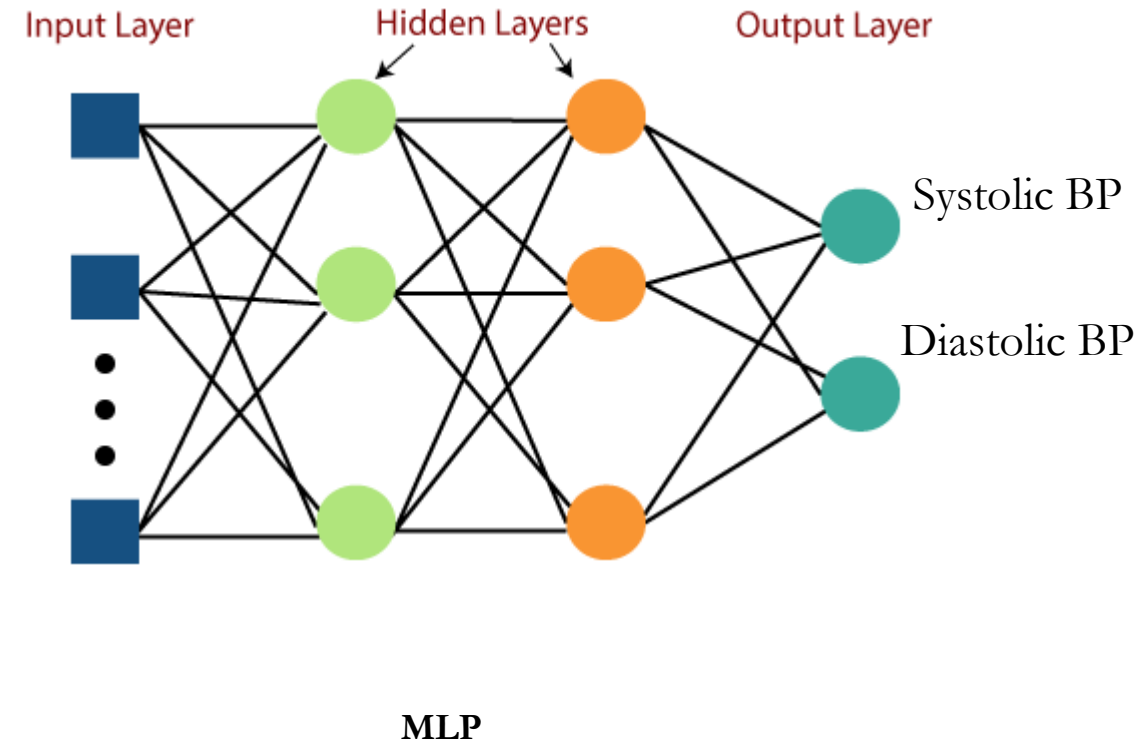


MLP

Sélection des hyperparamètres

(Règle générale)

L'approche générale repose sur l'utilisation d'un jeu de données de validation (prochain cours)



Type d'apprentissage (par instance, par mini-batch, par batch)

- **Apprentissage par instance (apprentissage en ligne)** : Dans ce mode, le modèle est mis à jour un exemple d'entraînement à la fois. Chaque point de données individuel est utilisé pour mettre à jour immédiatement les paramètres du modèle. (Par exemple, dans le dernier exercice du cours précédent, nous avons utilisé l'apprentissage par instance.)
- **Apprentissage par batch (lots)** : Dans ce mode, le modèle est mis à jour en utilisant l'ensemble des données en une seule fois. En d'autres termes, vous parcourez l'ensemble des données, calculez l'erreur et les gradients, puis mettez à jour les paramètres du modèle. Cette méthode est généralement coûteuse en termes de calcul pour les grands ensembles de données.
- **Apprentissage par mini-batch (mini-lots)** : Au lieu de mettre à jour le modèle une instance à la fois ou d'utiliser l'ensemble des données, un mini-lot de M échantillons est utilisé pour mettre à jour le modèle. Cela permet de trouver un équilibre entre l'efficacité des calculs et la fréquence de mise à jour du modèle.

Type d'apprentissage (par instance, par mini-batch, par batch)

Apprentissage par instance (apprentissage en ligne) (incrémental)

Quand l'utiliser ? Lorsque vous avez **un flux continu de données entrantes**, une mémoire limitée ou que des mises à jour en temps réel sont nécessaires.

Principal avantage : Permet au modèle de s'adapter rapidement aux changements dans la distribution des données.

Apprentissage par lots

À utiliser : Lorsque vous disposez d'un petit ensemble de données ou que les ressources informatiques ne sont pas un problème, et que vous souhaitez obtenir des paramètres de modèle stables et précis.

Principal avantage : Convergence **plus précise** vers un minimum puisque le modèle est mis à jour sur l'ensemble des données, mais cela a un coût de calcul élevé pour les grands ensembles de données.

Apprentissage par mini-lots

Quand l'utiliser : **Le plus souvent utilisé**, en particulier lorsque vous avez un ensemble de données modérément important et que vous cherchez un équilibre entre efficacité de calcul et vitesse de convergence.

Principal avantage : Combine certains avantages de l'apprentissage par lots et de l'apprentissage en ligne, offrant une convergence plus rapide et des mises à jour plus stables que l'apprentissage en ligne.

Itérations vs époques

Les termes "itérations" et "époques" sont souvent utilisés dans le contexte de l'entraînement des modèles d'apprentissage automatique, en particulier des réseaux neuronaux. Voici une explication concise de la différence :

Époque : Une époque correspond à un passage complet de tous les exemples d'entraînement de l'ensemble de données. En d'autres termes, au cours d'une époque, le modèle a vu et appris de chaque échantillon de l'ensemble de données une fois.

Itération : Une itération fait référence à une seule mise à jour des poids du modèle, généralement à l'aide d'un sous-ensemble de données (qui peut aller d'un seul échantillon (Apprentissage par instance) jusqu'à l'ensemble des données en entier (Apprentissage par lots)). Le nombre d'itérations dépend à la fois du nombre d'échantillons dans votre ensemble de données et de la taille du lot que vous utilisez pour l'entraînement.

Exemple : Si vous disposez d'un ensemble de données contenant 1000 échantillons et que vous choisissez une taille de mini-lot de 100, alors, une époque est composée de $1000/100 = 10$ itérations.

Si j'utilise l'apprentissage par lots, chaque itération est égale à une époque.

Si j'utilise l'apprentissage par instance, chaque époque correspond à 1000 itérations.

En résumé, une époque est composée d'une ou plusieurs itérations, en fonction de la taille du lot et de la taille de l'ensemble de données.

Résumé

- Définition de l'IA, de l'apprentissage automatique (ML) et de l'apprentissage profond (DL)
- Terminologies de l'apprentissage automatique
- Régression linéaire
- Algorithmes d'optimisation
- Perceptron multicouche (MLP)
- Algorithme de rétropropagation
- Sélection des hyperparamètres
- Type d'apprentissage (par instance, par mini-batch, par batch)

Prochain cours

- **Concepts fondamentaux de l'IA**
 - Classification vs Régression
 - Fonctions du Coût
 - Apprentissage supervisé et non supervisé
 - Overfitting et Underfitting
 - Augmentation des données
 - Déséquilibre des classes
 - Normalisation des données
- **Optimisation du modèle et sélection des hyperparamètres**
 - Entraînement, validation et test
 - Hold-Out Validation (Validation simple)
 - K-fold cross-validation (Validation croisée K-fold)
 - Éviter le surajustement
- **Analyse statistique et évaluation des performances**
 - Métriques de régression
 - Métriques de classification